

Einführung in die Programmiersprache C

Version vom 01.12.2024

Inhaltsverzeichnis

1	Erste Schritte	4
1.1	Installationsanleitung Microsoft Visual Studio 2022 Professional	4
1.2	Ein erstes Programm	4
1.3	C und C++	4
2	Grundaufbau eines C-Programms	5
3	Variablen	6
3.1	Variablennamen	6
3.2	Ganzzahl Datentypen: int, long & short	6
3.3	Gleitkommazahlen: float & double	7
3.4	Boolesche Variablen: bool	9
3.5	Konstanten	9
4	Bildschirm-Ein- und -Ausgabe	10
4.1	Ausgabe	10
4.2	Färbige Ausgabe	11
4.3	Eingabe von Werten	12
5	if-Anweisung	14
5.1	switch-Anweisung	18
6	Schleifen	19
6.1	for-Schleife	19
6.2	Verschachtelte for-Schleife	20
6.3	while-Schleife	20
6.4	do-while-Schleife	22
6.5	break und continue	22
6.6	Aufgaben zu Schleifen	23
7	Bildschirmsteuerung	25
8	Zufallszahlen	26
9	Typkonversion	28
9.1	Implizites Umwandeln	28
9.2	Explizite Typumwandlung	28
10	Datentyp char	30
10.1	Zeichenarten	30
10.2	Funktionen für Zeichen	32
11	Debugging von Programmen	36
12	Unterprogramme	38
12.1	Unterprogramme ohne Parameter	38
12.2	Unterprogramme mit Übergabeparameter	39
12.3	Unterprogramme mit Rückgabeparameter	40
12.4	Rekursive Funktionen	43
13	Zeiger bzw. Pointer	45
13.1	Funktionen mit Zeigern	46

13.2	Unterschiede zwischen Ein- und Ausgangsparemtern	46
13.3	Aufgaben zu Funktionen mit Zeigern	47
14	Felder bzw. Arrays.....	49
14.1	Eindimensionales Feld (Vektor)	49
14.2	Mehrdimensionales Feld (Matrix)	51
14.3	Felder als Übergabeparameter.....	51
14.4	Aufgaben zu Feldern	52
15	Strings (Zeichenketten).....	56
15.1	Strings sind Felder	56
15.2	Deklaration von Zeichenketten	56
15.3	Arbeiten mit Zeichenketten.....	56
15.4	Sonstige Bibliotheksfunktionen für Strings und Characters	57
15.5	Übergabe von Strings an Unterprogramme	57
15.6	Aufgaben zu Strings	58
16	Enumeration (Aufzählungen)	60
17	Strukturen	61
17.1	Deklaration von Strukturen	61
17.2	Zugriff auf Strukturen	61
17.3	Übergabe von Strukturen an Unterprogramme	63
17.4	Aufgaben zu Strukturen.....	63
18	Dateizugriffe.....	65
19	Bitverarbeitung	68
19.1	Verknüpfungen von Bits	68
19.2	Schiebeoperationen.....	68
19.3	Beispiele mit Bitoperationen.....	69
19.4	Ein Bit abfragen	69
20	Dynamische Speicherverwaltung	70
20.1	Speicher anlegen und freigeben	70
20.2	Größe des Speichers verändern	71

1 Erste Schritte

1.1 Installationsanleitung Microsoft Visual Studio 2022 Professional

- Gehe zu
- <https://edufs.edu.htl-leonding.ac.at/>
- klick „FTP-Browser“
- mit dem Schuluser anmelden.
- Klick Software/!Programmiertools
- [https://edufs.edu.htl-leonding.ac.at/ftp/Software/!Programmiertools%20\(IDEs%20und%20SDKs\)/Visual%20Studio%202022%20Enterprise/](https://edufs.edu.htl-leonding.ac.at/ftp/Software/!Programmiertools%20(IDEs%20und%20SDKs)/Visual%20Studio%202022%20Enterprise/)
- [VisualStudioSetup.exe](#) ausführen
- Der Produktschlüssel findet sich in [key.txt](#)
- »Desktopentwicklung mit C++« auswählen

1.2 Ein erstes Programm

- Visual C++ 2022 (oder 2019) starten
- Option »Neues Projekt erstellen« wählen
- Option »Konsolen-App C++« wählen
- Speicherort und Namen eingeben

Ein erstes Programm: „Hello World“ erscheint dann in Visual Studio:

```
// Test.cpp: Hauptprojektdatei.  
#include <iostream>  
  
int main()  
{  
    std::cout << "Hello World!\n";  
}
```

Zum Starten des Programms auf den grünen Pfeil klicken oder F5 drücken.

1.3 C und C++

C und C++ sind zwei unterschiedliche Programmiersprachen, wobei C++ vereinfacht erklärt eine Erweiterung von C ist. Das heißt alle Befehle in C werden auch in C++ funktionieren (aber umgekehrt nicht).

Das Ziel in diesem Fach ist es, C zu lernen. Dennoch verwenden wir der Einfachheit halber eine Umgebung mit der C++ auch programmiert werden kann. Das heißt, der *Compiler* (der Übersetzer, der unsere Programme so übersetzt, dass der Computer sie ausführen kann) ist ein C++ Compiler. Die Endung unserer Programme ist auch .cpp und nicht .c wie für C-Programme üblich.

Das erste Programm aus dem vorigen Abschnitt ist ein C++ Programm, da es einige Sprachelemente nutzt, die nicht in C vorhanden sind. Die folgende Aufgabe enthält das Programm, wie es in C aussieht.

Aufgabe

Schreibe das Programm Test.c aus dem vorigen Abschnitt um und führe es aus:

```
#include <stdio.h>  
  
void main(void)  
{  
    printf("Hello world!\n");  
}
```

2 Grundaufbau eines C-Programms

Beispielprogramm:

```
#include <stdio.h>    // Hier wird ein Programmteil inkludiert,
// damit das Programm Befehle wie z. B. printf() versteht.
// #include ist ein sogenannter Präprozessorbefehl

void main(void)
{
    // Am Anfang und Ende der Funktion main müssen
    // geschweifte Klammer stehen
    // Das ist ein Kommentar (zwei Schrägstriche)
    /* Das ist auch ein Kommentar, der über
       mehrere Zeilen gehen kann          */

    int k, nummer, wert; // Definition der Variablen
                        // ; heisst Semikolon oder Strichpunkt
    printf("Beispiel mit Variablen\n"); // Ausgabeanweisung
    printf("=====\n\n");

    k = 5;              // Initialisierung: k erhält den Wert 5
    printf("k = %d\n", k); // Ausgabe
    nummer = 6;         // = nennt man den Zuweisungsoperator
    wert = k + nummer;   // erste Berechnung
    printf("wert = %d\n", wert);
}
```

Dateiendungen:

- .c für C-Programme (.cpp für C++-Programme)
- .h für Header-Datei

Mit .h-Dateien kann man andere Programmteile/Bibliotheken einbinden. Das Einbinden erfolgt mit #include, z. B. #include <math.h>.

3 Variablen

Zum Rechnen benötigt man Variablen wie Radius r und Fläche a usw.

Eine Variable besitzt:

- Name
- Datentyp
- Wert
- Speicherplatz bzw. Adresse

3.1 Variablennamen

Variablennamen beginnen mit einem Buchstaben oder Unterstrich, gefolgt von Buchstaben, Ziffern oder einem Unterstrichen.

Nicht erlaubt:

- reservierte Namen (`main`, `int`, `float`, `for`, `if` usw.)
- Umlaute, β , das Leerzeichen und Sonderzeichen

Es wird zwischen Groß- und Kleinschreibung unterschieden (engl. *case sensitive*)! Beispiel: `value`, `Value` oder `VALUE` sind unterschiedliche Variablen.

Es wird empfohlen, den sogenannten *camel case* zu verwenden:

- Der Variablenname beginnt mit einem Kleinbuchstaben
- Zusätzliche Wörter beginnen mit einem Großbuchstaben

Man kennt das von manchen Markennamen (`iPhone`, `eBay`, usw.). Beispiele: `newValue`, `isOk`, `readOldValue`.

Beispiele: `r`, `R`, `radius`, `U_1`, `U_2`, `fläche`, `_123`, `*$`, `U-3`, `meßbereich`

3.2 Ganzzahl Datentypen: `int`, `long` & `short`

Beginnen wir mit `int`: `int` ist die Abkürzung für engl. *integer*, was so viel wie ganze Zahl heißt.

Definition bzw. Deklaration: `int varName;`

Die Definition muss immer vor der Verwendung bzw. Initialisierung geschehen. Die Definition der Variablen erfolgt gleich zu Beginn eines Programms.

Definition und Initialisierung:

```
int varName;           // Definition
varName = 10;          // Initialisierung
```

Definition und Initialisierung können auch kombiniert werden:

```
int varName = 10;      // Definition & Initialisierung
```

Es können auch mehrere Variablen in einer Zeile definiert und initialisiert werden:

```
int x, long y = 10, int a, b;
```

Es wird aber empfohlen, zwecks Übersichtlichkeit jeder Variable eine eigene Zeile zu gönnen.

Für die Zuweisung von ganzen Zahlen können die Zahlenformate dezimal, oktal und hexadezimal verwendet werden:

```
x = 619;           // dezimal
x = 0x1AF;         // hexadezimal, beginnt mit 0x
x = 0157;          // oktal, beginnt mit Null (0), wenn möglich vermeiden
```

Neben `int` gibt es auch noch die Datentypen `long` und `short`. Der Unterschied liegt in der Speichergröße. Diese ist leider nicht eindeutig festgelegt. Während beim PC ein `int` meist 4 Bytes groß ist, ist er bei einem Microcontroller oft nur 2 Bytes groß. Beim PC sind in den meisten Fällen `int` und `long` gleich groß.

Steht bei der Variablendefinition außer dem Datentyp nichts mehr dabei, so kann die Zahl positiv und negativ sein. Möchte man nur positiven Zahlen (also natürliche Zahlen mit 0) verwenden, so kann man das Schlüsselwort `unsigned`

benutzen: `unsigned int`, `unsigned long`, `unsigned short`. (Man könnte bei Zahlen mit Vorzeichen auch `signed` verwenden; besser jedoch weglassen, damit der Code lesbar bleibt.)

Je nach Speichergröße und vorzeichenbehaftet/vorzeichenlos ergeben sich unterschiedliche Zahlenbereiche. Zahlen mit Vorzeichen werden im Zweierkomplement gespeichert. Beim PC ergeben sich folgende Zahlenbereiche:

Datentyp	Speichergröße in Bytes	Minimum	Maximum
short	2	-32768 ($= -2^{15}$)	32767 ($= 2^{15}-1$)
unsigned short		0	65535 ($= 2^{16}-1$)
int	4	-2147483648 ($= 2^{31}$)	2147483647 ($= 2^{31}-1$)
unsigned int		0	4294967295 ($= 2^{32}-1$)
long	4	wie int	
unsigned long		wie unsigned int	

Um die Größe einer Variable per Programm zu bestimmen, wird der `sizeof`-Operator verwendet. Z. B. gibt der Aufruf `sizeof(int)`; oder auch `sizeof(varName)`; gleich 4 zurück.

Es lassen sich auch die Minimum- und Maximumwerte per Programm ermitteln. Dazu die Header-Datei `limits.h` einbinden (`#include <limits.h>`). Anschließend stehen die Konstanten `INT_MIN`, `INT_MAX`, `UINT_MAX`, `LONG_MIN` usw. zur Verfügung.

Operatoren

Mit einem Operator können die Grundrechenarten gerechnet werden. Folgende binäre Operatoren können auf ganze Zahlen angewandt werden: `+`, `-`, `*`, `/` (Ganzzahldivision), `%`. Der `%`-Operator macht eine Modulo-Division, das Ergebnis ist der Rest einer Ganzzahldivision. Beispiel:

```
x = 17 / 5; // ergibt die Ganzzahl 3 (nicht 3.4!)
y = 17 % 5; // ergibt 2 Rest
```

Die Zuweisung einer Variable erfolgt wie bei der Initialisierung mit dem `=` Operator:

```
x = 5; // die Variable x erhält den Wert 5
x = 6 + 2; // x wird zu 8
x = x + 2; // x wird um 2 erhöht und ist jetzt 10
```

Wird mit einer Variable gerechnet und das Ergebnis wieder in dieser Variable gespeichert, dann kann man abkürzend zusammengesetzte Operatoren verwenden:

```
a += 5; // entspricht a = a + 5;
b -= 3; // entspricht b = b - 3;
c *= 2; // entspricht c = c * 2;
d /= 3; // entspricht d = d / 3;
e %= 10; // entspricht e = e % 10;
```

Zur Abkürzung gibt es für das Addieren/Subtrahieren von 1 eine noch kürzere Schreibweise:

```
n++; // entspricht n = n + 1; bzw. n += 1;
m--; // entspricht m = m - 1; bzw. m -= 1;
```

Das Addieren von 1 nennt man Inkrementieren; das Subtrahieren von 1 Dekrementieren. `++` bzw. `--` nennt man auch Inkrement- bzw. Dekrementoperatoren.

3.3 Gleitkommazahlen: float & double

Oft reicht es nicht, nur Ganzzahl Datentypen zu verwenden. Möchte man Kommazahlen speichern, gibt es dafür die Datentypen `float` und `double`. Gespeichert werden die einzelnen Bits im Datentyp-Standard IEEE754.

Datentyp	Speichergröße in Bytes	Wertebereich	Genauigkeit, signifikante Stellen
float	4	ca. $\pm 3 \cdot 10^{\pm 38}$	ca. 7
double	8	ca. $\pm 1.7 \cdot 10^{\pm 308}$	ca. 16

Hinweis: Mit der Genauigkeit bei z. B. float ist gemeint, dass die ersten 7 Stellen der Zahl korrekt und unterscheidbar sind. Z. B. sind die float Zahlen 2345.2345 und 2345.2341111 nicht mehr unterscheidbar.

Als Dezimaltrennzeichen muss in C der Punkt (.) anstelle des Kommas (,) verwendet werden. Die Initialisierung erfolgt gleich wie bei Ganzzahl-Datentypen, z. B. float varName = 123.45;. Um bei Berechnungen eindeutig zu zeigen, dass es sich um eine Kommazahl handelt, muss zumindest ein Dezimalpunkt oder die exponentielle Notation (z. B. 10e3) vorkommen. Beispiele:

```
float wrong = 17 / 5;           // gibt 3 zurück, obwohl es ein float ist
float right1 = 17.0 / 5.0;      // gibt 3.4 zurück
float right2 = 1.7e1 / 5e0;     // gibt auch 3.4 zurück
```

Obwohl float und int denselben Speicher belegen, sollten für Ganzzahl-Rechnungen nur Ganzzahl-Datentypen verwendet werden. Diese sind immer exakt und die Grundrechenarten benötigen deutlich weniger Speicher bzw. Berechnungszeit.

Bis auf den Modulo-Operator funktionieren bei den Gleitkommazahlen dieselben Operatoren: +, -, *, /. Es lässt sich auch hier sizeof anwenden. Um die Minimum- bzw. Maximumwerte per Programm zu ermitteln, benötigt man die Header-Datei float.h. Dadurch lassen sich die Konstanten FLT_MIN bzw. FLT_MAX verwenden.

Funktionen

Die meisten mathematischen Funktionen benötigen die Headerdatei math.h:

Funktionen	Beispiele	Anmerkung
Trigonometrische Funktionen	sin(), cos(), tan()	Winkel müssen in Radiant angegeben werden!
Trigonometrische Umkehrfunktionen	asin(), acos(), atan()	
Exponential, Logarithmen	exp(), log(), log10()	log() ist der natürliche Logarithmus, log10() der Zehnerlogarithmus
Potenz, Wurzel	pow(x, r), sqrt()	pow(x, r) rechnet x^r sqrt() steht für <i>square root</i> = Quadratwurzel
Runden	ceil(), floor()	ceil() rundet auf nächste Ganzzahl auf, floor() rundet ab

Diese Funktionen rechnen mit double-Genauigkeit. Falls float-Genauigkeit ausreicht, dann gibt es die meisten Funktionen mit „f“ hinten, z. B. asinf().

In math.h sind auch bereits einige Konstanten hinterlegt. Dazu muss **vor** dem Einbinden von math.h eine Zeile eingefügt werden:

```
#define _USE_MATH_DEFINES
#include <math.h>
```

Anschließend stehen Konstanten wie π (M_PI), e (M_E) oder $\sqrt{2}$ (M_SQRT2) zur Verfügung.

Bei den folgenden Beispielen sollen float-Wert mit der Tastatur eingegeben werden. Dazu müssen wir etwas auf das nächste Kapitel vorgreifen. Um einen float einzulesen, verwenden Sie bitte die Funktion scanf_s() und zum Ausgeben die Funktion printf():

```
#include <stdio.h>

void main(void)
{
    float var;
    scanf_s("%f", &var);
    printf("%f\n", var);
}
```

Beide können mittels der Bibliothek stdio.h verwendet werden. Achte darauf, dass beim scanf_s ein kaufmännisches UND (&) vor der Variable steht.

Aufgabe 1: Kreis

Von einem Kreis mit dem Durchmesser d sind Umfang und Fläche zu berechnen. Der Durchmesser ist in mm einzugeben. In dem nachfolgenden Struktogramm ist der Ablauf des Programms skizziert:

Einlesen Zahl
$U = d \cdot \pi$
$A = d^2 \cdot \pi / 4$
Ausgabe U, A

Aufgabe 2: Rechteck

Berechne die Fläche und den Umfang eines Rechtecks mit den Seiten a und b .

Aufgabe 3: Widerstandsberechnung

Berechne den Gesamt Widerstand einer Serien- und einer Parallelschaltung zweier Widerstände.

Aufgabe 4: Rechtwinkeliges Dreieck

Lesen Sie die Seiten a und b eines rechtwinkligen Dreiecks ein und bestimme die Seite c , sowie die Winkel α , β und γ und geben Sie diese am Bildschirm aus.

3.4 Boolesche Variablen: bool

`bool` ist ein Datentyp, der die Werte `true` (wahr) oder `false` (nicht wahr) annehmen kann. In C sind Boolesche Variablen nicht standardmäßig integriert, man benötigt die Bibliothek `stdbool.h`.¹ Eine `bool`-Variable verhält sich wie eine `int`-Variable, der Wert für `false` ist gleich 0, der Wert für `true` ist gleich 1. Bei logischen Bedingungen (siehe später) wird nur unterschieden, ob die Variable 0 (`false`) oder **ungleich 0** (`true`) ist.

Beispiel:

```
bool flag = true;
printf("%d, ", flag);
flag = false;
printf("%d\n", flag);
```

Ausgabe: 1, 0

3.5 Konstanten

Möchte man bei einer Variablen die Eigenschaft zuweisen, dass sie zur Laufzeit nicht mehr geändert werden kann, so kann man das Schlüsselwort `const` vor die Variable stellen. Beispiel:

```
const float phi = 1.618034; // goldener Schnitt
```

Natürlich könnte man ja die Variable einfach nicht mehr verändern. `const` ist trotzdem sinnvoll, weil man damit anderen Programmieren (oder sich selbst) zeigt, dass es sich um eine Konstante handelt.

Eine zweite Möglichkeit, um Konstanten zu definieren, ist die Präprozessoranweisung `#define`:

```
#define PHI 1.618034
```

Hier braucht man kein `=`-Zeichen und auch keinen Strichpunkt. Alle Anweisungen, die mit `#` (engl. *hash*) beginnen, heißen Präprozessoranweisungen (*prä/pre* von gr. *vor*). Solche Anweisungen werden vor dem eigentlichen Programm analysiert. Durch das `#define` werden einfach alle Stellen, an denen im Programm später `PHI` vorkommt, durch die Konstante 1.618034 ersetzt. Es ist kein Muss, aber es hat sich eingebürgert, dass man `#define`-Konstanten in Großbuchstaben schreibt.

¹ In C++ ist `bool` bereits ein Standard-Datentyp.

4 Bildschirm-Ein- und -Ausgabe

4.1 Ausgabe

Mit der Funktion `printf()` (Bibliothek: `stdio.h`) können Zeichenketten, sogenannte *strings*, formatiert am Bildschirm ausgegeben werden.

Syntax: `printf("Kontrollstring", var1, var2, var3, ...);`

Beispiel:

```
#include <stdio.h>

void main(void)
{
    int num;
    num = 4;
    printf("Das ist das %d. Kapitel!", num);
}
```

Ausgabe: Das ist das 4. Kapitel!

Das `%d` im obigen Kontrollstring ist ein sogenanntes Formatelement (bzw. Formatierungszeichen), welches festlegt, in welcher Art und Weise die Variable `num` ausgegeben wird (d ... *decimal*, geeignet für `int`-Variablen).

Für jede Variable, die über `printf()` ausgegeben wird, muss im Kontrollstring ein Formatelement vorhanden sein. Formatelemente beginnen immer mit `%`. Formatelemente sind leider nicht standardisiert. Es können beliebig viele Variablen ausgegeben werden.²

Die wichtigsten Formatelemente

Element	Abkürzung für	Verwendung
<code>%d</code>	decimal	Ausgabe von Ganzzahlen: <code>short</code> , <code>int</code> , <code>long</code>
<code>%u</code>	unsigned (decimal)	Ausgabe von positiven ganzen Zahlen: <code>unsigned short</code> usw.
<code>%f</code>	float	Ausgabe eines <code>float</code>
<code>%lf</code>	long float	Ausgabe eines <code>double</code> , je nach Compiler oft gleich wie <code>%f</code>
<code>%e</code>	exponential	Ausgabe eines <code>float</code> in exponentieller Schreibweise, z. B. <code>1.234e3</code>
<code>%E</code>	exponential	Gleich wie <code>%e</code> , nur mit großem E, z. B. <code>1.234E3</code>
<code>%g</code> bzw. <code>%G</code>	?	Ausgabe eines <code>float</code> , verwendet die kürzere der beiden Formatierungen <code>%f</code> und <code>%e</code>
<code>%c</code>	character (=Zeichen)	Ausgabe von ASCII-Zeichen, siehe Kapitel 0
<code>%s</code>	string	Ausgabe von Text, z. B. <code>printf("Hello %s\n", "world!");</code>
<code>%o</code>	octal	Ausgabe von Ganzzahlen im oktalen Zahlensystem
<code>%x</code> bzw. <code>%X</code>	hexadecimal	Ausgabe von Ganzzahlen im hexadezimalen Zahlensystem (mit Klein- bzw. Großbuchstaben)
<code>%p</code>	pointer	Ausgabe der Adresse eines Pointers, siehe Kapitel 13

Da die Formatelemente mit `%` beginnen, kann damit kein `%`-Zeichen ausgegeben werden. Zur Ausgabe des `%`-Zeichens selbst ist `%%` zu verwenden.

Die Formatelemente können auch um die Angabe der Kommastellenanzahl erweitert werden.

Beispiel:

```
printf("Radius = %.3f mm \n", radius);
```

Gibt die Variable `radius` auf 3 Kommastellen genau aus.

² Solche Art von Funktionen nennt man *variadic functions*. In der Bibliothek `stdarg.h` sind dazu Hilfsfunktionen vorhanden. Weil das doch relativ aufwändig ist, schreibt man solche Funktionen nur selten selbst.

Weitere Optionen

Element	Beispiel	Verwendung
+	%+d	Gibt ein führendes + bei positiven Zahlen aus
.zahl	%.2f	Angabe der Nachkommastellen, siehe oben
zahl.zahl	%8.2f	1. Zahl: Gesamtzahl der Stellen (inkl. Dezimalpunkt); falls sich die vorgegebene Zahl nicht ausgeht, wird sie ignoriert 2. Zahl: wie .zahl
zahl	%10d	Gesamtzahl der Stellen, der Rest wird mit Leerzeichen aufgefüllt (praktisch für rechtsbündige Ausgabe von Zeilen)
0zahl	%08X	wie zahl, jedoch wird der Rest mit 0en aufgefüllt

Neben diesen Formatierungsoptionen gibt es noch sogenannte Escape Sequenzen, die jeweils mit \ (engl. *backslash*) beginnen. Die wichtigsten Escape Sequenzen:

- \n Zeilenvorschub, engl. *new line*, eine Zeile nach unten
- \r Wagenrücklauf, engl. *carriage return*, zurück zum Zeilenanfang
- \t Tabulator, je nach System z. B. 8 Zeichen breit

Die Escape Sequenzen kommen noch aus Zeiten der Schreibmaschinen. Bei den meisten Systemen ist heute \n\r kombiniert, sodass ein Aufruf von \n genügt, um eine neue Zeile zu beginnen.

Beispiele

```
int i = 431;
double x = 1000.0 / 7;

printf(">%5d<\n", i); // > 431<
printf(">%05d<\n", i); // >00431<
printf(">%04x<\n", i); // >01af<
printf(">%e<\n", x); // >1.428571e+02<
printf(">%2.10e<\n", x); // >1.4285714286e+02< *
printf(">%20.10e<\n", x); // > 1.4285714286e+02<
```

Die Pfeile >< dienen nur zur Anzeige der Grenzen.

*) Bei diesem Beispiel wird die erste Längenangabe 2 ignoriert.

Ausgabe von Umlauten

Bei deutschsprachigen Windows-Rechnern lassen sich die Umlaute wie folgt ausgeben:

Zeichen	Befehl
ä	\204
Ä	\216
ö	\224
Ö	\231
ü	\201
Ü	\232
ß	\341

Beispiel: `printf("Oberfl\204che = %d", surface);`

4.2 Färbige Ausgabe

Die Schriftfarbe kann unter Windows mit folgendem Befehl verändert werden:

```
#include <windows.h>
SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), 4); // red
```

Farbe	Nummer
Schwarz	0
Blau	1
Grün	2
Cyan	3

Rot	4
Magenta	5
Braun	6
Hellgrau	7
Dunkelgrau	8
Hellblau	9
Hellgrün	10
Hellrot	12
Hellmagenta	13
Gelb	14
Weiß	15
Blinken	128

4.3 Eingabe von Werten

Mit der Funktion `scanf_s()` (Bibliothek: `stdio.h`) können Zeichen, Zeichenketten und Zahlen vom Bildschirm eingelesen werden. Sie ist also quasi das Gegenteil von `printf()`. Die Formatelemente von `printf()` können auch hier verwendet werden. Benutze jedoch nie ein `\n`; die Eingabe wird automatisch mit Enter abgeschlossen.

Beispiel 1:

```
#include <stdio.h>

void main(void)
{
    float radius;

    printf("Gib einen Radius in mm ein\n");
    scanf_s("%f", &radius);
}
```

Beachte, dass vor der Variable unbedingt ein kaufmännisches UND (&) steht!³ Ein Vergessen dieses &-Operators führt zu einem Fehler beim Kompilieren.

Beispiel 2:

```
#include <stdio.h>

void main(void)
{
    int year;

    printf("Welches Jahr haben wir gerade?\n");
    scanf_s("%d", &year);
}
```

Es könnten auch bestimmte Zeichen in `scanf_s()` angegeben werden, welche dann exakt so eingegeben werden müssen. Praktisch reichen die Formatelemente von `printf()`.

Achtung! Beim Einlesen von `double` bzw. `long`-Variablen muss das Kontrollzeichen `l` (für *long*) vorangesetzt werden, daher `%lf` bzw. `%ld`. Das geht auch beim `printf()`; dort ist es aber nicht unbedingt notwendig.

Beispiele:

```
int x, y;
char text[11];           /* string aus 10 Zeichen + 0 am Ende */
char ch;
double z1;
long z2;

scanf_s("%c", &ch, 1);   /* 1 einzelnes Zeichen */
printf("Zeichen: %c\n", ch);
scanf_s("%10s", text, 11); /* strings mit max. 10 Zeichen */
printf("Text: %s\n", text);
scanf_s("%d%d", &x, &y); /* 2 Ganzzahlen mit beliebigen */
                        /* Zwischenraumzeichen getrennt */
```

³ Später werden wir lernen, dass mit dem & ein Zeiger (engl. *pointer*) übergeben wird. Das braucht man, damit die Funktion die übergebene Variable ändern kann.

```
printf("Zahlen: %d, %d\n", x, y);
scanf_s("%d%d", &x, &y);      /* 2 Ganzzahlen durch ? getrennt */
printf("Zahlen: %d, %d\n", x, y);
scanf_s("%lf", &z1);          /* 1 double */
printf("double: %f\n", z1);
scanf_s("%ld", &z2);          /* 1 long */
printf("long: %d\n", z2);
```

Vorsicht: `scanf_s()` bricht die Eingabe sofort ab, wenn die eingegebenen Zeichen nicht mehr den Formatangaben entsprechen!

Profitipp: Beim Einlesen einer Zahl kann die Eingabetaste im Eingabepuffer hängen bleiben und bei einem nachfolgenden `scanf_s()`-Befehl gelesen werden. Dieses unerwünschte Verhalten kann folgendermaßen abgestellt werden:

```
while (getchar() != '\n'); // leert den Input Stream
```

Mehr Details zum Befehl `scanf_s()` siehe auch Kapitel 15.

5 if-Anweisung

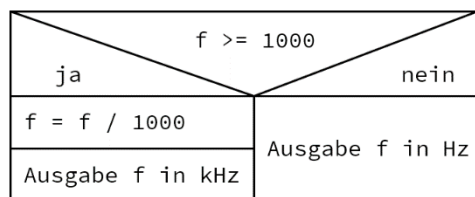
Mit der `if`-Anweisung sich Programm-Verzweigungen erstellen. Der `if`-Teil wird ausgeführt, wenn die Bedingung in Klammern zutrifft (d. h. ungleich 0 ist, siehe `bool`-Variablen weiter unten); ansonsten wird der `else`-Teil ausgeführt.

Syntax:

```
if (Bedingung)
{
    // Anweisung(en)
}
else
{
    // Anweisung(en)
}
```

Hinweise: die Bedingung muss immer in runden Klammern stehen. Nach `if` und `else` kommt **kein** Strichpunkt. Die Anweisungen stehen jeweils in `{}`.

Falls nicht benötigt, kann der `else`-Teil auch einfach weggelassen werden. Hier ein Beispiel für eine `if`-Anweisung mit dazugehörigem Struktogramm (`f` ist ein `float`):



```
if (f >= 1000.0)
{
    f /= 1000.0;
    printf("f = %.3f kHz\n", f);
}
else
{
    printf("f = %.3f Hz\n", f);
}
```

Das `>=` nennt man Vergleichsoperator. Diese Vergleichsoperatoren gibt es:

Operator	Bedeutung
<	kleiner
>	größer
<=	kleiner gleich
>=	größer gleich
==	genau gleich
!=	ungleich

Bitte den Vergleichsoperator `==` nie mit dem Zuweisungsoperator `=` verwechseln! Wird `=` in der `if` verwendet, wird einfach ein neuer Wert zugewiesen und dann auf ungleich 0 geprüft. Beispiel:

```
int x = 3;

if (x = 2) /* x wird 2 zugewiesen und 2 ist ungleich 0, also wahr */
{
    printf("%d\n", x); // gibt 2 aus
}
```

Möchte man prüfen, ob eine Zahl `x` ungleich 0 ist, kann man `if (x != 0)` verwenden. Es funktioniert aber auch folgendes:

```
if (x)
{
    printf("x ist ungleich 0\n");
}
```

Das geht deshalb, weil alle Bedingungen ungleich 0 gleich wahr sind, siehe Boolesche Variablen oben.

Wenn bei der `if`- bzw. `else`-Anweisung jeweils nur eine Zeile folgt, dann kann man auch die `{}`-Klammern weglassen:

```
if (x)
    printf("x ist ungleich 0\n");
```

Es wird aber empfohlen, auch bei nur einer Zeile immer `{}` zu schreiben. Manchmal hat nämlich ein Programm zunächst nur eine Anweisung. Danach möchte man noch eine 2. Anweisung einfügen und das führt dann zu Fehlern. Beispiel:

```
if (x)
    x++;
    printf("x\n"); // kein Syntax-Fehler, aber wird immer ausgeführt!
```

Zusammengesetzte Vergleiche

Möchte man z. B. prüfen, ob eine Zahl in einem bestimmten Bereich liegt, dann kann man Vergleiche auch kombinieren. Als Beispiel soll geprüft werden, ob $3 < x \leq 7$ ist. Dazu müssen die beiden Vergleiche $x > 3$ und $x \leq 7$ logisch UND-verknüpft werden. Das logische UND-Verknüpfen von Vergleichen macht man mit zwei UND-Zeichen `&&`:

```
if ((x > 3) && (x <= 7)) { /* ... */ }
```

Beachte die zusätzlichen runden Klammern zum Gruppieren der Vergleiche. Das ist zwar nicht notwendig, da die Vergleichsoperatoren immer vor den logischen Operatoren ausgewertet werden. Es erleichtert aber die Lesbarkeit des Programms.

Der Compiler macht folgendes: es wird überprüft, ob $x > 3$; dieser Ausdruck gibt `true` (1) oder `false` (0) zurück. Dann wird $x \leq 7$ ausgewertet. Die beiden Ergebnisse werden logisch `&&`-verknüpft (Wahrheitstabelle) und die `if`-Anweisung verzweigt schließlich je nach Ergebnis der `&&`-Verknüpfung.

Beachte, dass zusammengesetzte Vergleiche nicht direkt kombiniert werden können. Geht nicht:

```
if (3 < x <= 7) { /* <- geht nicht! */ }
```

Es gibt folgende Operatoren für logische Verknüpfungen:

Operator	Bedeutung
<code>&&</code>	logisches UND
<code> </code>	logisches ODER*
<code>!</code>	logisches NICHT

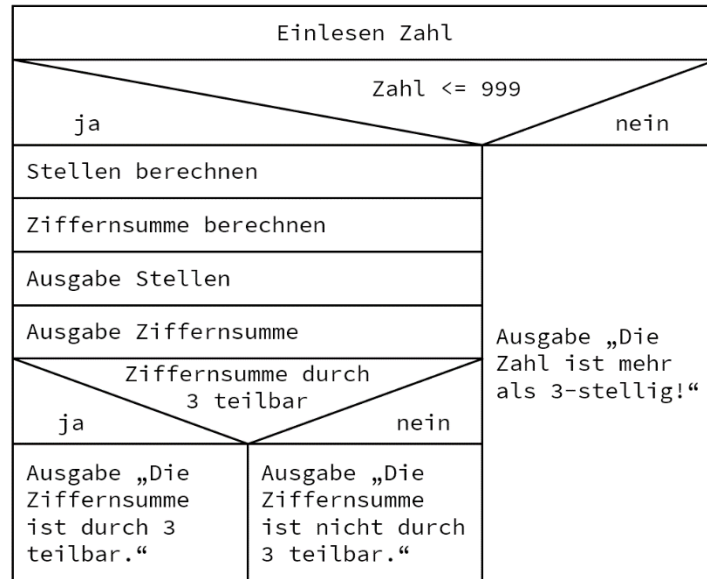
^{*)} die vertikale Linie erhält man durch Drücken von `Alt Gr + <`.

Beispiel: Ziffernsumme

1. Lese eine ganze Zahl mit maximal drei Stellen ein
2. Prüfe, ob die Zahl auch nicht mehr als 3-stellig ist und gib gegebenenfalls eine Meldung aus
3. Gib die 3 Stellen aus: Hunderter, Zehner, Einer
4. Gib die Ziffernsumme dieser Zahl aus
5. Prüfe, ob die Ziffernsumme ganzzahlig durch 3 teilbar ist und gib eine entsprechende Meldung aus

Die 1er-Stelle kann man ermitteln, indem man die Zahl Modulo 10 rechnet. Danach ganzzahlig durch 10 teilen und wieder Modulo 10 ergibt die nächste Stelle usw.

Struktogramm:



Programm:

```
#include <stdio.h>

void main(void)
{
    unsigned int number;
    unsigned int hundreds, tens, units, crossSum;

    printf("Gib eine max. 3-stellige Zahl ein:\n");
    scanf_s("%d", &number);

    if (number <= 999)
    {
        units = number % 10;
        number /= 10;
        tens = number % 10;
        number /= 10;
        hundreds = number % 10;
        crossSum = hundreds + tens + units;

        printf("Hunderter:    %d\n", hundreds);
        printf("Zehner:      %d\n", tens);
        printf("Einer:        %d\n", units);
        printf("Ziffernsumme: %d\n", crossSum);

        if ((crossSum % 3) == 0)
        {
            printf("Die Ziffernsumme ist durch 3 teilbar.\n");
        }
        else
        {
            printf("Die Ziffernsumme ist nicht durch 3 teilbar.\n");
        }
    }
    else
    {
        printf("Die Zahl ist mehr als 3-stellig!\n");
    }
}
```

Verschachtelte if-Anweisung

if-Anweisungen können beliebig verschachtelt werden. Beispiel:

```
if (x > 0)
{
    ; /* Code für x > 0 */;
}
```



```

else
{
    if (y < 0)
    {
        ; /* Code für x <= 0 && y < 0 */
    }
    else
    {
        ; /* Code für x <= 0 && y >= 0 */
    }
}

```

Erfolgt innerhalb vom else wieder eine if, so kann auch direkt else if verwendet werden. Beispiel:

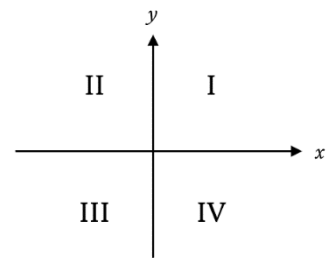
```

if (x > 10)
{
    ; /* Code für x > 10 */;
}
else if (x > 5)
{
    ; /* Code für x <= 10 && x > 5 */
}
else
{
    ; /* Code für x <= 5 */
}

```

Aufgabe 1: Quadrantenbestimmung

Lies die x - und y -Koordinaten eines Punktes ein (Datentyp `int`) und bestimme, in welchem Quadranten sich der Punkt befindet. Weiters sollen die Sonderfälle Koordinatenursprung (0, 0), auf der x -Achse bzw. auf der y -Achse erkannt werden.



Aufgabe 2: Dreiecksbestimmung

Schreibe ein Programm, das aus drei eingelesenen Seitenlängen feststellt, ob diese Strecken ein Dreieck bilden und wenn dies der Fall ist, ob das Dreieck gleichseitig, gleichschenkelig, rechtwinkelig oder allgemein ist.

Aufgabe 3: Notenbestimmung

Schreibe ein Programm, das Testergebnisse in Noten umrechnet:

Punkte	Note
0 – 49	Nicht genügend
50 – 62	Genügend
63 – 75	Befriedigend
76 – 88	Gut
89 – 100	Sehr Gut

Eingabe: Prozentpunkte; Ausgabe: Note

5.1 switch-Anweisung

Mit einer switch kann einfach mehrere Möglichkeiten geprüft werden. Es ist also ein Kurzform für if ... else if ... else if ... else if ... else.

Syntax:

```
switch (variable)
{
    case const1:
        ; /* Code für variable == const1 */
        break;
    case const2:
        ; /* Code für variable == const2 */
        break;
    case const3:
    case const4:
        ; /* Code für variable == const3 */
        /* oder variable == const4 */
        break;
    case const5:
        ; /* Code für variable == const5 */
        break;
    default:
        ; /* Code für "else" */
}
```

Anders als bei der if-Anweisung dürfen bei einer switch nur ganzzahlige Ausdrücke verwendet werden (int ja, float nein). Beachte, dass nach jedem Fall (case) ein break kommen muss. Ansonsten geht das Programm einfach in die nächste Zeile. Möchte man mehrere Fälle zusammenfassen, so kann man das break aber auch absichtlich weglassen, siehe const3 und const4 in der Syntax.

Aufgabe: Wochentag

Schreibe ein Programm, das eine positive ganze Zahl einliest. Mit einer switch soll unterschieden werden, welchem Wochentag diese Zahl entspricht. 1 ... Montag, 2 ... Dienstag, usw. Wird eine Zahl >7 eingegeben (default), soll eine Fehlermeldung erscheinen.

6 Schleifen

6.1 for-Schleife

Die for-Schleife wird verwendet, wenn vorher schon klar ist, wie oft die Schleife durchlaufen werden soll.

Syntax:

```
for (Startanweisung; Schleifenbedingung; Update)
{
    ; /* Schleifen-Code */
}
```

- *Startanweisung*: wird 1x vor der Schleife ausgewertet
- *Schleifenbedingung*: die Schleife wird ausgeführt, solange diese Bedingung true ist
- *Update*: wird bei jedem Schleifendurchlauf ausgeführt

Achte wieder genau auf die Zeichen: zwischen den Anweisungen/Bedingungen kommt ein Strichpunkt (;), nach der Klammer kommt KEIN Strichpunkt (so wie bei if). Wie bei der if-Anweisung könnten bei nur einer Zeile Code in der Schleife auch die Klammern {} weggelassen werden. Es wird aber wie immer empfohlen, welche zu verwenden.

Beispiel:

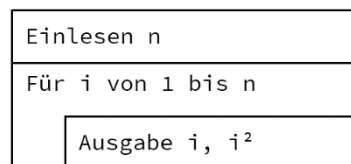
```
int i;

for (i = 1; i <= 10; i++)
{
    printf("%d\n", i);
}
```

Ausgabe: Zahlen von 1 bis 10.

Aufgabe 1

Schreib eine for-Schleife, die die Zahlen von 1 bis n und ihre Quadratzahlen 1, 4, 9, usw. ausgibt. n soll eingelesen werden. Struktogramm:



Mit integer $n \geq 1$:

1. Ausgabe gerade Zahlen bis n
2. Ausgabe 7er Reihe bis $7 \cdot n$
3. Countdown von n hinunter auf 0
4. Summe der Zahlen bis n ausgeben
5. Turmrechnung für eine gegebene Zahl n bis 9 und retour: $n \cdot 2$, Ergebnis $\cdot 3$, Ergebnis $\cdot 4$, usw.

Beispiel für $n = 3$:

- $3 \cdot 2 = 6$
 - $6 \cdot 3 = 18$
 - $18 \cdot 4 = 72$
 - ...
 - $120960 \cdot 9 = 1088640$; danach rückwärts, also
 - $1088640 : 2 = 544320$
 - $544320 : 3 = 181440$
 - ...
 - $27 : 9 = 3$
6. Bumm-Spiel, z. B. mit 6er Reihe: jedes Mal, wenn die Ziffer 6 in der Zahl vorkommt oder die Zahl durch 6 teilbar ist, soll anstelle der Zahl »bumm« ausgegeben werden.
Basis-Variante: $n \leq 99$. Bonus *: $n \leq 999$. Bonus **: $n \leq \text{INT_MAX}$.

Aufgabe 2: Zahlenfolgen

Gib folgende Zahlenfolgen (20 Elemente) aus:

1. ungerade Zahlen: 1, 3, 5, 7, 9, ...
2. 2er Potenzen: 1, 2, 4, 8, 16, ...
3. Fibonacci-Zahlen: 1, 1, 2, 3, 5, 8, 13, ... (die nächste Zahl ist die Summe der beiden vorhergegangenen Zahlen)
4. Summe: zeige, dass die Summe wie bei Aufgabe 1.4 auch mit $i * (i + 1) / 2$ berechnet werden kann
5. Kehrwerte: 1, 0.5, 0.3333, 0.25, 0.2, ...
6. Alternierende: 1, -1, 2, -2, 3, -3, 4, -4, ...

Aufgabe 3: Anzahl Durchläufe

Wie oft werden folgende Schleifen durchlaufen? Variablen jeweils vom Datentyp `int`:

```
for (k = 1; k < 5; k++) {}  
for (m = 3; m > 0; m--) {}  
for (k = -2; k <= 3; k++) {}  
for (u = 0; u < 5; u--) {}  
for (k = 1; k > -2; k--) {}  
for (;;) {}
```

Aufgabe 4: Trigonometrie

Teste, ob die Beziehung $\sin(2x) = 2\sin(x)\cos(x)$ für alle x stimmt. Mache dazu eine `for`-Schleife, die x von 0 bis 2 in Schritte von 0.2 durchläuft (also `float` oder `double`), berechne beide Seiten der Beziehung, gib sie am Bildschirm aus und vergleiche die beiden Ausgaben am Bildschirm.

Aufgabe 5: Sechseck

Berechne den Umfang von einem Sechseck, indem die 6 Seitenlängen mit Hilfe einer `for`-Schleife eingelesen und aufsummiert werden.

6.2 Verschachtelte for-Schleife

Eine verschachtelte Schleife liegt vor, wenn eine Schleife eine weitere Schleife beinhaltet. Dies ist unabhängig vom Schleifentyp möglich.

Beispiel: Ausgabe des »Kleinen 1 × 1«

```
int i, j;  
for (i = 1; i <= 10; i++)  
{  
    for (j = 1; j <= 10; j++)  
    {  
        printf("%d x %d = %d\n", i, j, i * j);  
    }  
    printf("\n");  
}
```

6.3 while-Schleife

Eine `while`-Schleife wird dann verwendet, wenn im Vorherein nicht bekannt ist, wie oft die Schleife durchlaufen wird; z. B. wenn die Schleife durch eine Bedingung abgebrochen werden soll, die erst während eines Schleifen-Durchlaufs berechnet wird.

Syntax:

```
while (Schleifenbedingung)  
{  
    ; /* Schleifen-Code */  
}
```

Beispiel:

```
int i = 0;

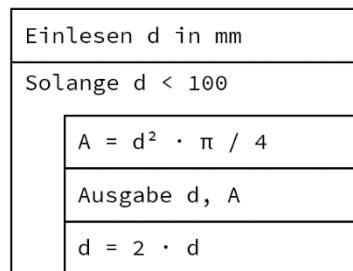
while (i < 3)
{
    printf("%d\n", i);
    i++;
}
```

Ausgabe: 0, 1, 2. Dieses Beispiel könnte man auch mit einer for-Schleife machen, weil die Bedingung recht klar ist. Oft kann man Aufgaben auch durch beide Schleifen gut lösen und es ist mehr oder weniger Geschmacksache. Beim nächsten Beispiel ändert sich die Bedingung durch den Inhalt der Schleife.

Beispiel: while

Lese einen Durchmesser d vom Datentyp float ein und berechne die zugehörige Kreisfläche. Wiederhole diese Aufgabe jeweils mit verdoppeltem Durchmesser, solange der Durchmesser kleiner als 100 mm ist.

Struktogramm:



Programm:

```
#define _USE_MATH_DEFINES
#include <math.h>
#include <stdio.h>

void main(void)
{
    float d;
    float A;

    scanf_s("%f", &d);

    while (d < 100.0)
    {
        A = d * d * M_PI / 4;
        printf("d = %8.3f mm, A = %8.3f mm2\n", d, A);
        d *= 2;
    }
}
```

Die while-Schleife prüft vor dem ersten Durchgang schon die Schleifen-Bedingung. Es kann also auch vorkommen, dass die Schleife kein einziges Mal durchlaufen wird.

Beispiel: Endlosschleife

Bei den Beispielen mit for-Schleife haben wir bereits eine Endlosschleife kennen gelernt. Noch einfacher geht's mit einer while-Schleife:

```
while (1)
{
    ; /* endlos Code */
}
```

Hier wird wieder der Trick verwendet, dass alle Bedingungen ja insgeheim auf ungleich 0 prüfen.

Man könnte meinen, eine Endlosschleife sei sinnlos, weil danach kein Code mehr ausgeführt werden kann. Bei Windows-Programmen wie hier stimmt das auch. Aber sobald mal Code für einen Microcontroller schreibt, braucht

man eigentlich immer eine Endlosschleife. In dieser werden bis zum Ausschalten des Geräts die Anweisungen durchgeführt.

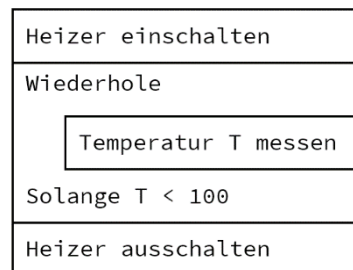
6.4 do-while-Schleife

Eine do-while-Schleife ist sehr ähnlich der while-Schleife. Der einzige Unterschied besteht darin, dass erst am Ende eines Durchlaufs geprüft wird, ob die Schleifenbedingung wahr ist. Die do-while-Schleife wird also auf jeden Fall zumindest 1× durchlaufen.

Syntax:

```
do
{
    ; /* Schleifen-Code */
} while (Schleifenbedingung);
```

Als Beispiel für eine do-while-Schleife hier das Struktogramm eines einfachen Temperaturreglers:



Aufgabe 1: do while

Lese eine positive ganze Zahl kleiner als 10 ein. Verdopple die Zahl und gib sie aus. Wiederhole die Verdopplung und Ausgabe, solange die Zahl kleiner als 1000 ist. Skizziere auch ein passendes Struktogramm.

Aufgabe 2: Überprüfung der Eingabe A

Es ist ein Durchmesser vom Datentyp `int` einzulesen. Es sollen nur Durchmesser ab 5 mm und kleiner als 25 mm erlaubt sein. Falls die Eingabe außerhalb dieses Bereichs liegt, soll sie wiederholt werden (ohne Nachricht). Bei korrekter Eingabe soll die Eingabe ausgegeben werden.

Aufgabe 3: Überprüfung der Eingabe B

Erweitere das vorherige Programm, sodass bei falscher Eingabe eine Hinweis-Nachricht erscheint. Skizziere ein passendes Struktogramm.

Aufgabe 4: Kreisberechnung, noch einmal

Lese den Radius eines Kreises mit Datentyp `float` ein. Berechne immer wieder die Fläche des Kreises, solange ein positiver Radius eingegeben wird. Beende das Programm, sobald 0 oder ein negativer Wert eingegeben wird.

6.5 break und continue

Mit `break` kann eine Schleife vorzeitig abgebrochen werden. `continue` bricht *einen* Schleifendurchlauf ab und macht beim nächsten Durchlauf weiter. `break` und `continue` funktionieren in jeder Art Schleife.

Beispiel mit break: hier wird max. 5× eine ganze Zahl eingelesen und die Summe berechnet. Sobald eine negative Zahl eingegeben wird, bricht die Berechnung ab.

```
#include <stdio.h>

void main(void)
{
    int i;
    int number;
    int sum = 0;

    for (i = 1; i <= 5; i++)
    {
        printf("Gib Zahl %d ein: ", i);
        scanf_s("%d", &number);

        if (number < 0)
        {
            break;
        }

        sum += number; // sum = sum + number;
    }

    printf("Summe = %d", sum);
}
```

Beispiel mit continue: dasselbe Beispiel, nur mit continue statt break. Die Eingabe einer negativen Zahl bricht die Berechnung nicht ab, sondern ignoriert diesen einen Durchlauf.

```
#include <stdio.h>

void main(void)
{
    int i;
    int number;
    int sum = 0;

    for (i = 1; i <= 5; i++)
    {
        printf("Gib Zahl %d ein: ", i);
        scanf_s("%d", &number);

        if (number < 0)
        {
            continue;
        }

        sum += number; // sum = sum + number;
    }

    printf("Summe = %d", sum);
}
```

6.6 Aufgaben zu Schleifen

Aufgabe 1: Reihenberechnung

Berechne mit einem C-Programm folgende Summen:

1. $\sum_{k=1}^{100} \frac{k}{k^2+1}$
2. $\sum_{m=1}^{75} \frac{0,5^m}{m^2}$
3. $\sum_{n=1}^{90} \frac{0,9^n}{n\sqrt{n+1}}$

Hinweis: falls die Schleife mit `int` durchlaufen werden, dann muss für die Summe auf `float` oder `double` umgerechnet werden. Das funktioniert, indem man `(float) k` bzw. `(double) k` bei der Berechnung schreibt. Details dazu folgen in Kapitel 9.

Ergebnisse: 1.) 4.515560, 2.) 0.582241, 3.) 1.258026

Aufgabe 2: Reihe für $\ln(2)$

Berechne mit einem C-Programm folgende Summe:

$$\sum_{n=1}^{10} \frac{1}{n \cdot 2^n}$$

Diese Reihe konvergiert gegen $\ln(2)$. Führe die Berechnung als `double` durch und gib das Ergebnis auf 15 Kommastellen aus. Gib als Vergleich dazu die Konstante `M_LN2` ebenfalls auf 15 Kommastellen aus. Erhöhe dann solange das Ende der Schleife, bis die Reihe gleich genau wie die Konstante ist. Wie oft muss aufsummiert werden?

Erweiterung: lass die Schleife bis maximal 50 Durchläufe machen. Füge eine Bedingung ein, die die Schleife abbricht, sobald die Summe auf 12 Kommastellen genau bei `M_LN2` liegt.

Aufgabe 3: Produktberechnung

Berechne mit einem C-Programm folgende Produkte:

1. $\prod_{k=1}^{50} \frac{1.2k}{k+1}$
2. $\prod_{m=1}^{20} \frac{\sqrt{m+2}}{2}$

Ergebnisse: 1.) 178.439964, 2.) 22608.324868

Aufgabe 4: Römische Zahlen

Schreibe ein Programm, welches eine (arabische) Zahl in römische Zahlen umwandelt. Das Programm soll den Benutzer um eine ganze Zahl zwischen 1 und 3999 fragen und diese Zahl in die entsprechende römische Zahl umrechnen.

Die römischen Ziffern lauten:

römisch	dezimal
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Beachte die Sonderfälle 4 (IV), 9 (IX), 40 (XL), 90 (XC), 400 (CD) und 900 (CM)! Das Programm soll solange ausgeführt werden, bis der Benutzer 0 eingibt.

Aufgabe 5: Summe und Mittelwert einer Zahlenfolge

Schreibe ein Programm, das die Summe und den Mittelwert einer Zahlenfolge berechnet. Die Eingabe wird mit der Zahl 0 abgebrochen.

Aufgabe 6: Notenspiegel

Es sollen die Noten zwischen 1 und 5 eingegeben werden. 0 bricht die Eingabe ab. Dann soll der Notenspiegel und die Durchschnittsnote ausgegeben werden. Verwende eventuell die `switch`-Anweisung.

7 Bildschirmsteuerung

Wir haben nun schon recht oft Ausgaben mit `printf()` gemacht. Manchmal ist es auch nützlich, den Bildschirm wieder zu löschen. Das erfolgt durch die Funktion `system()` aus der Bibliothek `stdlib.h`:

```
#include <stdlib.h>

void main(void)
{
    /* ... */
    system("cls"); /* Bildschirm löschen */
}
```

Mit der Hilfsfunktion `gotoxy()` ist es auch möglich, den Cursor an eine bestimmte Stelle zu setzen. Eine darauf folgende Ausgabe mit `printf()` gibt dann den Text beginnend an der neuen Cursor-Position aus. Um diese Funktion nützen zu können, kopiere den nachfolgenden Code zwischen die `#include ...` und dem Hauptprogramm:

```
void gotoxy(int x, int y)
{
    printf("%c[%d;%df", 0x1B, y + 1, x + 1);
}
```

Die linke obere Ecke ist die Position 0, 0. Ein Aufruf von `gotoxy(4, 2)`; setzt den Cursor auf das 5. Zeichen von links und die 3. Zeile von oben.

Aufgabe 1: Sterne

Gib das folgende »Bild« mit n Zeilen von Sternen aus ($1 \leq n \leq 20$). Beispiel mit $n = 5$:

```
*
**
***
****
*****
```

Für dieses Beispiel wird `gotoxy()` nicht benötigt, kann aber verwendet werden.

Aufgabe 2: Weihnachtsbaum

Gib einen Weihnachtsbaum mit n Zeilen aus ($1 \leq n \leq 20$). Beispiel mit $n = 4$:

```
  *
 ***
*****
*****
```

Dieses Beispiel ist vermutlich einfacher mit `gotoxy()` zu erstellen. Versuche, auch eine Variante ohne `gotoxy()` zu schreiben.

8 Zufallszahlen

Manchmal benötigt man Zufallszahlen. Beispiele: Spiele, Simulation von Signalen, Reduktion von Rauschen, usw. Leider kann man in C keine echten Zufallszahlen erzeugen; C erzeugt nur Pseudozufallszahlen, d. h. sie wiederholen sich nach einer bestimmten Zeit. Durch Einbinden der Bibliothek `stdlib.h` können Pseudozufallszahlen mit der Funktion `rand()` erzeugt werden.

Beispiel:

```
int i;
unsigned int random;

for (i = 0; i < 10; i++)
{
    random = rand();
    printf("%d\n", random);
}
```

Erzeugt 10 Zufallszahlen. Der Wertebereich dieser Zufallszahlen liegt zwischen 0 und 32767. Die obere Grenze ist auch als Konstante `RAND_MAX` definiert. Beachte, dass die Zufallszahlen gleichverteilt sind, d. h. jeder Wert von 0 bis 32767 ist gleich wahrscheinlich.

Zur Einschränkung des Zahlenbereichs bietet sich eine Modulo-Berechnung an. Möchte man zum Beispiel Würfelzahlen (1 bis 6) erzeugen, kann man das so lösen:

```
dice = (rand() % 6) + 1;
```

Allgemein kann man Zufallszahlen mit den Grenzen inklusive `min` und `max` folgendermaßen erzeugen:

```
int random;
int max = 20;
int min = 10;
random = (rand() % (max + 1 - min)) + min;
```

Das funktioniert für `max > min` auch mit negativen Zahlen. Beachte, dass der Bereich `(max-min)` möglichst viel kleiner als 32767 ist.

Man kann auch Zufallszahlen vom Datentyp `float` erzeugen:

```
float random;
random = (float) rand() / RAND_MAX;
```

Aufgabe 1: Roulette A

Schreib ein Programm, dass 20× Zufallszahlen für ein Roulette-Spiel ausgibt, also Zahlen zwischen 0 und 36.

Wenn man ein Programm mit Zufallszahlen mehrmals laufen lässt, dann kommen immer dieselben »Zufallszahlen« vor. Um dies zu verhindern, gibt es die Funktion `srand(zahl)`. Damit kann man den Startwert für das Erzeugen der Zufallszahlen ändern.

Aufgabe 2: Roulette B

Lass das Programm Roulette ein paar Mal laufen und beobachte, dass immer dieselben Zufallszahlen ausgegeben werden. Füge nun am Programmbeginn `srand(100)` hinzu. Lass das Programm wieder ein paar Mal laufen. Wie verhalten sich die Zufallszahlen jetzt?

Um jedes Mal komplett neue Zufallszahlen zu erzeugen, müsste `srand()` auch mit einer Zufallszahl gefüttert werden. Da wir eine solche aber nicht haben, wird's schwierig. In der Praxis löst man solche Probleme, indem man z. B. die Zeit verwendet, seit der Computer läuft. Unter Windows (mit `windows.h`) kann folgendes benutzt werden:

```
#include <stdlib.h>
#include <stdio.h>
#include <windows.h>

void main(void)
{
    int i;
```

```

srand(GetTickCount());

for (i = 0; i < 20; i++)
{
    printf("%d\n", rand() % 37); // roulette
}
}

```

Die Funktion `GetTickCount()` liefert die Zeit in Millisekunden, seit Windows läuft. Das ist zwar auch keine Zufallszahl, aber diese Zeit hängt zumindest davon ab, wann das Programm »zufällig« gestartet wird.

Aufgabe 3: Würfelpoker

Simuliere mit einer Zufallszahl einen Würfel und würfle so lange, bis eine 6 gewürfelt wird.

Erweiterungen:

1. Anzahl der Würfel auf 2 erhöhen und solange würfeln, bis zwei 6er gewürfelt werden
2. Anzahl der Würfel auf 5 erhöhen und solange würfeln, bis ein 6er Grande (Grande = 5× gleiche Augenzahl) gewürfelt wird.

Ab hier: 5 Würfel, 100× würfeln.

3. Anzahl aller Granden ausgeben
4. Anzahl aller Poker (Poker = 4× gleiche Augenzahl) ausgeben
5. Anzahl Full House (3× gleich + 2× gleich) ausgeben
6. Anzahl kleiner Straßen (1, 2, 3, 4 oder 2, 3, 4, 5 oder 3, 4, 5, 6) ausgeben
7. Anzahl großer Straßen (1, 2, 3, 4, 5 oder 2, 3, 4, 5, 6) ausgeben

Aufgabe 4: Zahlenratespiel

Ermittle eine Zufallszahl zwischen 1 und 100. Der Benutzer soll die Zahl erraten:

- Wenn die eingegebene Zahl kleiner oder größer als die Zufallszahl ist, soll eine entsprechende Meldung ausgegeben werden und ein neuer Versuch gemacht werden können.
- Wenn die Zufallszahl erraten wurde, soll die Anzahl der Versuche ausgegeben werden.

9 Typkonversion

Das Umwandeln von einem Datentyp in einen anderen Datentyp heißt Typkonversion. Es wird unterschieden zwischen implizitem und explizitem Umwandeln. Das implizite Umwandeln macht der Compiler von sich aus. Beim expliziten Umwandeln gibt man klare Anweisungen, was umgewandelt werden soll.

9.1 Implizites Umwandeln

Beispiele für impliziertes Umwandeln:

```
int i, j, k;
float x, y, z;

i = 3;
j = 4;
x = 2.5;
y = -3.14159;

k = i + j;           // 3 + 4 = 7; OK
printf("%d\n", k);

z = x + y;           // 2.5 - 3.14159 = -0.61459; OK
printf("%f\n", z);

k = i + x;           // 3 + 2.5 = 5.5 -> k = 5
printf("%d\n", k);

z = i + x;           // 3 + 2.5 = 5.5 -> z = 5.5
printf("%f\n", z);
```

Was macht der Compiler? Die einzelnen Variablen oder Konstanten auf der rechten Seite vom `=`-Zeichen werden in den größten numerischen Datentyp der rechten Seite umgewandelt. Anschließend wird das Ergebnis in den Datentyp der linken Seite gewandelt.

Die »Rundung« von Fließkommazahlen beim impliziten Casting erfolgt unabhängig von den Nachkommastellen immer in Richtung 0. Genau genommen wird gar nicht gerundet, sondern nur die Nachkommastellen weggeschnitten.

Zu beachten ist, dass bei Operationen, die als Operanden zwei Werte desselben Typs haben, die Operation nur in diesem Typ durchgeführt wird und anschließend eventuell in einen anderen Typ konvertiert wird. Dies wird oft bei der Integerdivision vergessen:

```
int i, j, k;
double x;

i = 7;
j = 3;

k = i / j;           // 7/3 = 2 -> k = 2
printf("%d\n", k);

x = i / j;           // 7/3 = 2 -> x = 2.0, nicht 2.3333!
printf("%f\n", x);
```

Hier benötigen wir eine explizite Typumwandlung.

9.2 Explizite Typumwandlung

Der Wert einer Variablen kann explizit in einen Wert eines anderen Typs umgewandelt werden, indem man einfach den neuen Typ in Klammern vor den ursprünglichen Wert oder die Variable schreibt. Man nennt die runden Klammern um den Datentyp auch *cast*-Operator (*to cast* – gießen, betonieren). Man sagt auch auf Englisch »die Variable muss gecastet werden«. Beispiele:

```
int i, j, k;
double x, y;

i = 7;
j = 3;
y = 2.5;
```

```

x = (double) i + (double) j; // 7.0 + 3.0 = 10.0
printf("%f\n", x);

x = (double) (i + j); // 7 + 3 = 10 -> 10.0
printf("%f\n", x);

x = 3.5;
k = (int) x + (int) y; // 2 + 3 = 5 (ohne cast: 6)
printf("%d\n", k);

x = ((double) i) / j; // 7.0 / 3 = 2.3333 (ohne cast: 2)
printf("%f\n", x);

```

Die ersten beiden Rechnungen würden auch ohne *casten* dasselbe Ergebnis liefern. Bei den beiden unteren Beispielen würde ohne *casten* jedoch etwas anderes herauskommen.

Praxistipp: wenn man sich nicht sicher ist, ob die implizite Typkonversion richtig ist, dann lieber einmal zu viel als zu wenig explizit *casten*.

10 Datentyp char

char ist die Abkürzung von *character*. Beim Programmieren handelt es sich dabei um Zeichen. Mit einer char-Variable lässt sich 1 Zeichen speichern. Später werden wir mehrere Zeichen auf einen sogenannten *string* (Zeichenkette) zusammenfügen. Damit man Zeichen vom Code unterscheiden kann, werden diese unter einfache Anführungszeichen gesetzt, z. B. 'a', '1', usw.

10.1 Zeichenarten

- Buchstaben: 'a' bis 'z' und 'A' bis 'Z'
- Ziffern: '0' bis '9'
- Sonderzeichen: '!', '\$', '*', '#', usw.
- Trennzeichen: ' ' (Leerzeichen, *space*), '\t' (Tabulator)
- Steuerzeichen: '\n', '\b' (*backspace*), *enter*, usw.
- Spezielle Zeichen: '\\\ ' (das Zeichen \ selbst), '% ' (Prozentzeichen), '\ ' (Apostrophzeichen), '\" ' (Anführungszeichen)

Die Zeichen werden einfach als Zahl gespeichert, wobei jedem Zeichen eine eindeutige Zahl zugewiesen wird. Die einfachste Art der Zuweisung ist der sogenannte ASCII-Code (American Standard Code for Information Interchange). Hier werden 7 bit (0 bis 127) zur Codierung der wichtigsten Zeichen verwendet. Da moderne Computer immer Datentypen mit Vielfachen von 8 bit haben, wird das 8. bit (128 bis 255) für den *erweiterten* ASCII-Code verwendet. Dieser erweiterter Code ist jedoch nicht standardisiert. Je nach Weltregion wurden unterschiedliche Erweiterungen verwendet. In Westeuropa war z. B. der Standard ISO 8859-1 verbreitet, der die meisten Sonderzeichen westeuropäischer Sprachen beinhaltet (z. B. ä, ß, ö, å, ç, etc.).

Der ASCII-Code geht auf die 60er Jahre des vergangenen Jahrhunderts zurück. Er wird heute noch immer gerne bei einfachen Geräten verwendet. Auf PCs, Smart Phones usw. wird heute standardmäßig Unicode eingesetzt. Bei diesem Standard sind weit über 100 000 Zeichen definiert.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0 ^@ NUL	1 ^A SOH	2 ^B STX	3 ^C ETX	4 ^D EOT	5 ^E ENQ	6 ^F ACK	7 ^G BEL	8 ^H BS	9 ^I HT	10 ^J LF	11 ^K VT	12 ^L FF	13 ^M CR	14 ^N SO	15 ^O SI
	NULL	START OF HEADING	START OF TEXT	END OF TEXT	END OF TRANSM.	ENQUIRY	ACKNOWLEDGE	BELL	BACKSP.	CHARACT. TABULATION	LINE FEED	LINE TABULATION	FORM FEED	CARRIAGE RETURN	SHIFT OUT	SHIFT IN
1	16 ^P DLE	17 ^Q DC1	18 ^R DC2	19 ^S DC3	20 ^T DC4	21 ^U NAK	22 ^V SYN	23 ^W ETB	24 ^X CAN	25 ^Y EM	26 ^Z SUB	27 ^[ESC	28 ^\ FS	29 ^] GS	30 ^^ RS	31 ^_ US
	DATALINK ESCAPE	DEVICE CONTROL 1	DEVICE CONTROL 2	DEVICE CONTROL 3	DEVICE CONTROL 4	NEG. ACKNOWLEDGE	SYNCHRONOUS IDLE	END OF TRANSM.	CANCEL	END OF MEDIUM	SUBSTITUTE	ESCAPE	INFO. SEP. 4	INFO. SEP. 3	INFO. SEP. 2	INFO. SEP. 1
2	32 ^ SPACE	33 ^! EXCLAM. MARK	34 ^" QUOT. MARK	35 ^# NUMBER SIGN	36 ^\$ DOLLAR SIGN	37 %^ PERCENT SIGN	38 ^& AMPERSAND	39 ^' APOSTROPHE	40 ^(LEFT PAREN.	41 ^) RIGHT PAREN.	42 ^* ASTERISK	43 ^+ PLUS SIGN	44 ^, COMMA	45 ^- HYPHEN-MINUS	46 ^. FULL STOP	47 ^/ SOLIDUS
3	48 0	49 1	50 2	51 3	52 4	53 5	54 6	55 7	56 8	57 9	58 : COLON	59 ; SEMI-COLON	60 < LS.-THAN SIGN	61 = EQUALS SIGN	62 > GR.-THAN SIGN	63 ? QUESTION MARK
4	64 ^@ COMM'IAL AT	65 ^A	66 ^B	67 ^C	68 ^D	69 ^E	70 ^F	71 ^G	72 ^H	73 ^I	74 ^J	75 ^K	76 ^L	77 ^M	78 ^N	79 ^O
5	80 ^P	81 ^Q	82 ^R	83 ^S	84 ^T	85 ^U	86 ^V	87 ^W	88 ^X	89 ^Y	90 ^Z	91 ^[LEFT SQ. BRACKET	92 ^\ REVERSE SOLIDUS	93 ^] RT. SQ. BRACKET	94 ^_ CIRCUM'X ACCENT	95 ^` LOW LINE
6	96 ^ GRAVE ACCENT	97 ^a	98 ^b	99 ^c	100 ^d	101 ^e	102 ^f	103 ^g	104 ^h	105 ^i	106 ^j	107 ^k	108 ^l	109 ^m	110 ^n	111 ^o
7	112 ^p	113 ^q	114 ^r	115 ^s	116 ^t	117 ^u	118 ^v	119 ^w	120 ^x	121 ^y	122 ^z	123 ^{ L. CURLY BRACKET	124 ^ VERTICAL LINE	125 ^} R. CURLY BRACKET	126 ^~ TILDE	127 ^? DEL DELETE

Aufgabe

Welche Codenummer (dezimal bzw. hexadezimal) haben die Zeichen 'B', 'x' und '8'?

Wie bereits erwähnt verwendet man in C den Datentyp `char` für das Speichern von einzelnen Zeichen. Der Datentyp `char` ist genauso wie `short` oder `int` ein Ganzzahl-Datentyp, jedoch mit nur 8 bit. Man kann `char` bzw. `unsigned char` auch ganz normal zum Rechnen verwenden. Möchte man der Variable ein Zeichen zuordnen, dann muss man das Zeichen nur in einfache Anführungszeichen setzen. Beispiel:

```
char ch1, ch2;
ch1 = 'd'; // Buchstabe d
ch2 = '\\"'; // Anführungszeichen
```

Gibt man eine `char`-Variable per `%d` am Bildschirm aus, so erhält man den Code (also die Zahl) des Zeichens:

```
printf("%d\\n", ch1);
printf("%d\\n", ch2);
```

Ausgabe: 100 und 34.

Möchte man hingegen das Zeichen als Zeichen ausgeben, so ist `%c` zu verwenden:

```
printf("%c\\n", ch1);
printf("%c\\n", ch2);
```

Ausgabe: d und ".

Da Zeichen als Code bzw. Zahl gespeichert werden, können damit auch ganz normale `if`-Anweisungen durchgeführt werden. So kann man beispielsweise prüfen, ob ein Zeichen ein Kleinbuchstabe ist:

```
char ch;

/* ... */

if ((ch >= 'a') && (ch <= 'z'))
{
    printf("Das Zeichen ist ein Kleinbuchstabe.\\n");
}
```

Es wird ganz einfach 97 für 'a' bzw. 122 für 'z' verwendet und geprüft. Oder man könnte diesen Kleinbuchstaben auch in einen Großbuchstaben umwandeln:

```
char ch = 'e';

if ((ch >= 'a') && (ch <= 'z'))
{
    ch = ch + 'A' - 'a'; // Zeichen + 65 - 97, also Zeichen -32
    printf("Zeichen: %c\\n", ch);
}
```

Ausgabe: E.

Um ein einzelnes Zeichen von der Tastatur einzulesen, verwendet man am besten die Funktion `getchar()` (Bibliothek `stdlib.h`). Beispiel:

```
char ch;
ch = getchar();
printf("char: \\\"%c\\\"\\n", ch);
```

Aufgabe: ASCII-Tabelle

Schreibe ein Programm, dass alle ASCII-Zeichen von 0 bis 127 am Bildschirm ausgibt (1 Zeile pro Zeichen). Gib das Zeichen unter einfachen Anführungszeichen aus. Gib daneben (mit `\\t` getrennt) den Code als Dezimalzahl und daneben den Code als Hexadezimalzahl mit 2 Stellen und führender 0 (also z. B. 0A statt nur A) aus. Was fällt bei den Zeichen mit den Nummern 8, 10, 13 und 27 auf?

Beispiel: Noch einmal?

Am Ende eines Spiels soll gefragt werden, ob der Spieler noch einmal spielen will.

Lösung:

```
#include <stdio.h>

void main(void)
{
    char ch;

    do
    {
        /* Code für das Spiel */

        printf("Noch einmal spielen? [j]\n");
        do { scanf_s("%c", &ch, 1); } while (getchar() != '\n');
    } while (ch == 'j');
}
```

Die Zeile `do { scanf_s("%c", &ch, 1); } while (getchar() != '\n');` wird benutzt, um den Tastaturpuffer zu entleeren. Es kann passieren, dass nach dem Einlesen (z. B. einer ganzen Zahl) das Steuerzeichen für neue Zeile (`\n`) im Tastaturpuffer verbleibt. In diesem Fall würde ein `getchar()`-Befehl einfach übersprungen werden (um genau zu sein würde `getchar()` einfach das Steuerzeichen lesen) und die Abfrage daher nicht funktionieren.

10.2 Funktionen für Zeichen

Wir haben bereits einige Funktionen für Zeichen kennengelernt (mit `char ch`):

- Ausgabe eines Zeichens: `printf("%c", ch);`
- Einlesen eines Zeichens: `scanf_s("%c", &ch, 1);` bzw. `ch = getchar();`

Unter Verwendung der Bibliothek `conio.h` gibt es noch Varianten von `getchar()`:

- Einlesen eines Zeichens ohne Enter-Taste und ohne Bildschirmausgabe:

```
ch = _getch();
```

- Einlesen eines Zeichens ohne Enter-Taste aber mit Bildschirmausgabe:

```
ch = _getche();
```

Ein einzelnes Zeichen kann ausgegeben werden per (auch `conio.h`):

```
_putch(ch);
```

Durch Einbinden der Bibliothek `ctype.h` werden viele zusätzliche Funktionen für Zeichen zugänglich:

Funktion	Beschreibung
<code>ch = toupper(ch);</code>	Umwandlung auf einen Großbuchstaben (engl. <i>upper case</i>)
<code>ch = tolower(ch);</code>	Umwandlung auf einen Kleinbuchstaben (engl. <i>lower case</i>)
<code>isupper(ch);</code>	Prüfen, ob es ein Großbuchstabe ist
<code>islower(ch);</code>	Prüfen, ob es ein Kleinbuchstabe ist
<code>isalpha(ch);</code>	Prüfen, ob es sich um einen Buchstaben handelt
<code>isascii(ch);</code>	Prüfen, ob es sich um ein ASCII-Zeichen handelt
<code>isalnum(ch);</code>	Prüfen, ob es ein Buchstabe oder eine Ziffer ist (<i>alphanumeric</i>)
<code>isdigit(ch);</code>	Prüfen auf eine Ziffer (0, 1, ..., 9)
<code>isxdigit(ch);</code>	Prüfen auf eine Hex-Ziffer (0, 1, ..., 9, A, B, ..., F)
<code>ispunct(ch);</code>	Prüfen auf Satzzeichen (engl. <i>punctuation mark</i>)
<code>isspace(ch);</code>	Prüfen auf jede Art von Zwischenraum: Leerzeichen, <code>\n</code> , <code>\f</code> , <code>\r</code> , <code>\t</code> , <code>\v</code>
<code>isctrl(ch);</code>	Prüfen, ob es eine Steuerungstaste ist (<i>enter, delete, ...</i>)

Beispiele

Prüfen, ob 'j' oder 'J' gedrückt wurde:

```
if (toupper(ch) == 'J') { /* ... */ }
```


Prüfen, ob es sich um einen Großbuchstaben handelt:

```
if (isupper(ch)) { /* ... */ }
```

Eine weitere interessante Funktion ist `_kbhit()` (*keyboard hit*). Damit kann abgefragt werden, ob eine Taste gedrückt wurde, ohne das Programm zu unterbrechen und zu warten. Das ist vor allem für Spiele praktisch, bei denen der Spielablauf nicht unterbrochen werden soll.

Nachfolgend ein Beispiel mit `_kbhit()`. Es zeigt auch, wie die Cursor-Tasten (←→↑↓) abgerufen werden können. Dazu müssen unter Windows zwei Zeichen eingelesen werden. Das erste Zeichen hat den Wert `-32`, das zweite enthält den Code für die Cursor-Tasten. Das Programm wird mit Drücken der Escape-Taste beendet.

```
#include <conio.h>
#include <stdio.h>

void main(void)
{
    char ch;

    ch = 0;
    printf("Spiel l\204uft ...\n");

    do
    {
        /* spiel kommt hier her */

        if (_kbhit())
        {
            ch = _getch();          // gedrückte Taste einlesen

            /* hier können Zeichen abgefragt werden */

            if (ch == -32)          // Cursor beginnen mit -32
            {
                ch = _getch();

                if (ch == 77)
                {
                    printf("Cursor rechts\n");
                }
                else if (ch == 75)
                {
                    printf("Cursor links\n");
                }
                else if (ch == 72)
                {
                    printf("Cursor oben\n");
                }
                else if (ch == 80)
                {
                    printf("Cursor unten\n");
                }
            }
        }
    } while (ch != 27);           // 27 = ESC-Taste

    printf("Spiel Ende!\n");
}
```

Aufgabe 1: Zeichencharakterisierung

Schreibe ein Programm, das ein eingegebenes Zeichen charakterisiert. Lese dazu ein Zeichen ein. Gib wie beim Beispiel mit der ASCII-Tabelle das Zeichen selbst und den Code im dezimal- und hexadezimal-System aus. Gib weiters aus, ob es sich um eine Ziffer oder ein Satzzeichen handelt. Brich das Programm ab, sobald die Escape-Taste gedrückt wurde.

Aufgabe 2: Zeichenerkennung

Das Programm soll so lange Zeichen einlesen, solange alphanumerische Zeichen eingegeben werden. Am Ende soll dann die Anzahl der eingegebenen Ziffern und die Anzahl an Selbstlauten (a, e, i, o, u) ausgegeben werden.

Beispiel: Jäger

Auf dem Bildschirm mit 25 Zeilen (0 bis 24) und 80 Spalten (0 bis 79) bewegt sich ein Zeichen (Jäger). Der Jäger beginnt in der Mitte des Bildschirms und bewegt sich Schritt für Schritt (Zeichen für Zeichen) voran; zu Beginn von links nach rechts. Durch Einbau eines Pausenbefehls (Funktion `Sleep()` aus `windows.h`) soll die Bewegung langsam erfolgen.

Funktionsweise:

- Bei Bewegung nach rechts erhöht sich die x-Koordinate stets um 1 ($\Delta x = 1$), während die y-Koordinate gleich bleibt ($\Delta y = 0$).
- Eine neue Koordinate erhält man, indem man die alte Position löscht (ein Leerzeichen ausgibt) und die neue Position berechnet ($x = x + \Delta x$; $y = y + \Delta y$) und an der neuen Position wieder das Jäger-Zeichen ausgibt.
- Die Richtungssteuerung soll mit den Zifferntasten 8 (nach oben), 2 (nach unten), 4 (nach links) und 6 (nach rechts) erfolgen.

Nach 600 Schritten endet das Spiel.

Lösung:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <windows.h>

#define WIDTH 80          /* Bildschirmbreite */
#define HEIGHT 25        /* Bildschirmhöhe */

void gotoxy(int x, int y)
{
    printf("%c[%d;%df", 0x1B, y + 1, x + 1);
}

void main(void)
{
    const int maxCount = 600;    // Anzahl Schritte bis Spielende
    int i;                      // Schrittzähler
    int deltaX, deltaY;         // Änderung je Schritt
    int wait;                   // Wartezeit zwischen Schritten in ms
    int x, y;                   // Position des Jägers
    const char hunter = 219;    // ASCII-Zeichen für den Jäger
    char ch;                    // gedrückte Taste

    wait = 100;                 // Wartezeit zu Beginn
    x = WIDTH / 2;              // Startposition Bildschirmmitte
    y = HEIGHT / 2;
    deltaX = 1;                 // Startrichtung rechts
    deltaY = 0;

    gotoxy(x, y); printf("%c", hunter); // Jäger auf Startposition setzen

    for (i = 1; i <= maxCount; i++)
    {
        gotoxy(x, y);
        printf(" ");            // alte Position überschreiben

        /* neue Position berechnen */
        x = (x + deltaX) % WIDTH;
        y = (y + deltaY) % HEIGHT;

        if (x < 0)               // wenn linker Rand verlassen wird
        {
            x = WIDTH - 1;
        }
        if (y < 0)               // wenn rechter Rand verlassen wird
        {
            y = HEIGHT - 1;
        }

        gotoxy(x, y);
        printf("%c", hunter);    // Zeichen an neuer Position setzen
    }
}
```

```

Sleep(wait);                // Verzögerung der Bewegung

if (_kbhit())                // wurde eine Taste gedrückt?
{
    ch = _getch();
    if (ch == '8')           // nach oben
    {
        deltaX = 0;
        deltaY = -1;
    }
    else if (ch == '4')      // nach links
    {
        deltaX = -1;
        deltaY = 0;
    }
    else if (ch == '2')      // nach unten
    {
        deltaX = 0;
        deltaY = 1;
    }
    else if (ch == '6')      // nach rechts
    {
        deltaX = 1;
        deltaY = 0;
    }
}
}
}

```

Aufgabe 3: Erweiterung auf Jäger und Wolf

1. Erweitere das Programm um diagonale Bewegungen, also z. B. erfolgt die Bewegung nach Drücken der Taste '9' nach rechts oben.
2. Erstelle an einer Zufallsposition (0 bis WIDTH-1 bzw. 0 bis HEIGHT-1) einen Wolf durch Ausgabe des Zeichens '#'. Speichere die Koordinaten des Wolfs in den Variablen `xWolf` und `yWolf`.
3. Erwischt der Jäger den Wolf, dann soll der Wolf mit dem Jäger überschrieben werden. Verwende eine Treffer-Variable `hits` und zähle jeden Treffer nach oben.
4. Gib die Trefferzahl an der Position 1, 1 aus.
5. Erzeuge bei jedem Treffer einen neuen Wolf.
6. Reduziere die Wartezeit um jeweils 10 ms, falls die Leertaste gedrückt wird. Die minimale Wartezeit soll 10 ms betragen.
7. Reduziere die Wartezeit automatisch um 10 ms alle 5 Treffer.
8. Brich das Spiel vorzeitig ab, sobald Escape gedrückt wird.
9. Lass den Wolf mit halber Geschwindigkeit vom Jäger per Zufallsrichtung bewegen.
10. Platziere ein Hindernis '0' am Bildschirm. Wird dieses berührt, ist das Spiel verloren.

11 Debugging von Programmen

Debugging kommt von engl. *bug* – Käfer. Ein Bug ist ein Programmierfehler. Beim Debugging versucht man also, Bugs zu finden und diese auszumerzen. Dazu gibt es heute mächtige Werkzeuge.

Beim Schreiben von Programmen haben wir schon oft gesehen, dass Fehler aufgetreten sind. Der Compiler informiert in der Regel schon recht genau darüber, wo ein Fehler gemacht – z. B. ein Strichpunkt vergessen oder `printf()` geschrieben – wurde. Solche Fehler nennt man *Syntax*-Fehler. Es gibt aber auch logische Fehler. Diese nennt man auch *Semantik*-Fehler. Ein logischer Fehler wäre beispielsweise, wenn man eine `for`-Schleife einmal zu oft durchläuft. Das wird vom Compiler nicht erkannt, weil dieser ja nicht wissen kann, was das Programm machen soll.

Mit dem Debugger einer Programmierumgebung wie Visual Studio kann das Programm Schritt für Schritt analysiert werden. Das ist sehr hilfreich beim Auffinden semantischer Fehler.

Aufgabe

Es soll ein Roulette-Spiel simuliert werden. Dabei wird immer auf ungerade (fr. *impair*) gesetzt. Die Strategie des Spielers ist:

- Einsatz zu Beginn €100
- Falls ungerade: Gewinn und erneut Einsatz von €100 beim nächsten Spiel
- Falls nicht ungerade, also gerade oder 0: Verlust und doppelter Einsatz beim nächsten Spiel
- Beenden des Spiels, sobald der Spieler mindestens €2000 besitzt oder sobald der Spieler alles bis auf €100 verloren hat

Das folgende Programm enthält logische Fehler. Diese Fehler sind durch Debugging herauszufinden und zu korrigieren.

Programm:

```
#include <stdio.h>
#include <conio.h>
#include <windows.h>

#define MIN_CAPITAL 100
#define MAX_CAPITAL 2000

int main(void)
{
    int number;
    int game;
    int bet;
    int capital;

    game = 1;           // Spielzähler
    bet = 100;          // Einsatz zu Beginn in Euro
    capital = 1000;      // Gesamtkapital zu Beginn in Euro

    srand(GetTickCount());

    printf("Regel: ungerade gewinnt!\n");
    printf("Es wird gespielt, bis das Kapitel \
>=%d Euro bzw. <=%d Euro ist.\n", MAX_CAPITAL, MIN_CAPITAL);
    printf("Anfangskapital = %d Euro\n\n", capital);

    while ((capital < 2000) && (capital > 100))
    {
        number = rand() % 39; // Roulette Zufallsnummer erzeugen

        printf("Spiel Nr. %2d:\tEinsatz = %4d Euro, Roulettenummer = %2d\n",
            game, bet, number);

        if ((number < 0) || (number > 36))
        {
            printf("Fehlerhafte Zufallszahl!\n");
            break;
        }

        if ((number % 2) == 1) // Zufallszahl ungerade?
```

```

    {
        // JA -> gewonnen
        capital = capital + 2 * bet;
        bet = 100;
    }
    else
    {
        // NEIN -> verloren
        bet = bet * 2;           // Einsatz verdoppeln
    }

    printf("Neues Kapital = %4d Euro\n", capital);
    printf("Zum Weiterspielen bitte eine Taste drücken\n");
    //_getch();

    game++;
}

printf("Spiel beendet!\n");
}

```

Aufgabe

1. Kopiere dir das Programm in ein neues Projekt
2. Lass das Programm ein paar Mal laufen und beobachte, was passiert. Zum besseren Verständnis kann die Zeile mit `_getch()` nach belieben ein-/ausgeblendet werden.
3. Überwache folgende Variablen: `capital`, `bet`, `number`
4. Führe das Programm in Einzelschritten (F10) aus und beobachte die Variablenwerte
5. Setze einen Breakpoint im JA-Zweig der `if`-Anweisung. Führe das Programm so oft aus (F5), bis der Breakpoint einmal erreicht wird. Was ist falsch mit `number`? Korrigiere das Programm!
6. Was ist sonst noch semantisch falsch an dem Programm (2 Fehler)? Korrigiere diese Fehler und überprüfe den korrekten Ablauf des korrigierten Programms!
7. Entferne den Breakpoint

Bonusaufgabe:

Erweitere das Programm, indem 1000 Gesamtspiele mit Gewinn/Verlust simuliert werden. Mache eine Statistik, ob sich diese Spiel-Strategie langfristig lohnt.

12 Unterprogramme

12.1 Unterprogramme ohne Parameter

Unterprogramme sind vor allem dann nützlich, wenn bestimmte Teile des Programms immer wieder gebraucht werden. Wir haben schon ganz viele Unterprogramme verwendet; alle Funktionen sind nichts anderes als Unterprogramme. Diese haben wir bis jetzt immer durch Bibliotheken eingebunden. Es ist aber auch möglich, eigene Unterprogramme zu erstellen.

Bei einfachen Programmen können die Unterprogramme einfach über der Hauptfunktion `main()` geschrieben werden. Das machen wir jetzt. Bei komplexeren Projekten kann man sich auch selbst ganze Bibliotheken anlegen.

Beispiel: Sterne

Ein Unterprogramm soll n Sterne nebeneinander ausgeben, z. B. `*****` für $n = 5$. Als n soll eine *globale* Variable verwendet werden. Globale Variablen kommen gleich unter die Präprozessoranweisungen (`#...`). Sie sind für alle Unterprogramme, die darunter stehen, gültig. Wird eine Variable innerhalb eines Unterprogramms definiert, so nennt man das *lokale* Variable. Bisher haben wir nur lokale Variablen verwendet.

```
#include <stdio.h>

int n;           // globale Variable

void stars(void) // Unterprogramm: Deklaration & Definition
{
    int i;       // lokale Variable
    for (i = 0; i < n; i++)
    {
        printf("*");
    }
    printf("\n");
}

void main(void)
{
    n = 5;
    stars();           // Unterprogramm aufrufen
    n = 3;
    stars();           // Unterprogramm aufrufen
}
```

Die Reihenfolge ist bei C sehr wichtig. Ein Unterprogramm, welches aufgerufen wird, muss vor dem Aufruf schon deklariert (bekannt gemacht) werden. Man kann aber auch Deklaration und Definition trennen. Selbes Beispiel:

```
#include <stdio.h>

int n;           // globale Variable

void stars(void); // Unterprogramm Deklaration oder Prototyp

void main(void)
{
    n = 5;
    stars();           // Unterprogramm aufrufen
    n = 3;
    stars();           // Unterprogramm aufrufen
}

void stars(void) // Unterprogramm Definition, also mit Code
{
    int i;       // lokale Variable
    for (i = 0; i < n; i++)
    {
        printf("*");
    }
    printf("\n");
}
```

Das ist bei einer einzelnen Datei nicht unbedingt notwendig. Es macht das Programm aber ein wenig übersichtlicher, wenn die Unterprogramme erst nach `main()` stehen.

Wir haben hier eine globale Variable verwendet, weil das Unterprogramm ja sonst nicht weiß, wie groß n ist. In der Praxis sollte man globale Variablen vermeiden; es könnten zu viele globale Variablen bei Verwendung von Bibliotheken werden, sodass es unübersichtlich wird. Bei der Programmierung von Microcontrollern ist es trotzdem manchmal zweckmäßig (z. B. aus Gründen der Geschwindigkeit), globale Variablen zu verwenden.

12.2 Unterprogramme mit Übergabeparameter

Zur Vermeidung globaler Variablen kann man dem Unterprogramm auch Parameter übergeben.

Beispiel: Sterne, verbessert

```
#include <stdio.h>

void stars(int numStars)
{
    int i;
    for (i = 0; i < numStars; i++)
    {
        printf("*");
    }
    printf("\n");
}

void main(void)
{
    int n;
    n = 10;

    stars(5);
    stars(3);
    stars(n);
}
```

Dieses Beispiel verwendet einen Übergabeparameter. Möchte man mehrere Übergabeparameter einsetzen, dann sind diese einfach per Beistrich (,) zu trennen. Anders als bei Variablen-Deklarationen muss jedoch für jeden Übergabeparameter der Datentyp angegeben werden. Beispiel:

```
void function(float x, int y, short z) { /* Code */ }
```

Bei der Übergabe von Parametern wird eine Kopie des Parameter-Werts erstellt. Der übergebene Parameter kann durch die Funktion nicht verändert werden, nur die Kopie davon.

Aufgabe 1: Rechteck

Erweitere das bestehende Programm, um das Unterprogramm `void rectangle(int width, int height)`, das ein Rechteck aus Sternen zeichnet.

Aufgabe 2: Mittelwert

Schreibe ein Unterprogramm, das den Mittelwert von zwei `float`-Variablen ausgibt.

Aufgabe 3: Widerstandsberechnung

Schreibe jeweils ein Unterprogramm, welches den Serien- und Parallelwiderstand von zwei Widerständen berechnet und ausgibt.

Aufgabe 4: Maximum

Schreibe ein Unterprogramm, das aus zwei Werten den Maximalwert am Bildschirm ausgibt.

12.3 Unterprogramme mit Rückgabeparameter

Es können natürlich nicht nur Parameter an das Unterprogramm übergeben werden; ein Unterprogramm kann auch einen Parameter zurückgeben. Wie in Mathematik spricht man dann von Funktionen. Beim Programmieren versteht aber unter Unterprogramm und Funktion immer dasselbe.

Bei Unterprogrammen mit Rückgabeparameter ersetzt man einfach das `void` (»Nichts«-Datentyp) vor dem Funktionsnamen durch den Rückgabedatentyp.

Beispiel:

```
#include <stdio.h>

double absolute(double x)
{
    if (x < 0.0)
    {
        return -x;
    }
    else
    {
        return x;
    }
}

void main(void)
{
    double val = -10.0;
    double retVal;

    retVal = absolute(val);
    printf("%lf\n", retVal);
}
```

Die Funktion `absolute()` gibt den Betrag einer Zahl zurück. Die Funktion selbst wird beendet, indem `return` mit dem Rückgabeparameter aufgerufen wird.

Hinweis: auch Unterprogramme ohne Rückgabeparameter können `return` zum Beenden des Unterprogramms verwenden. Hier wird aber nichts zurückgegeben, also nur: `return`;

Man kann Rückgabeparameter genauso behandeln, als wären es normale Variablen oder Konstanten:

- Ganz normal rechnen:

```
retVal = absolute(val) + 10.0;
```

- Direkt in eine andere Funktion einsetzen:

```
printf("%lf\n", absolute(val));
```

Beispiel: Fakultät

Es soll eine Funktion geschrieben werden, die die Fakultät einer positiven ganzen Zahl n zurückgibt. Die Berechnung der Fakultät (mathematische Schreibweise: »! $\llcorner$ ») erfolgt mit der Formel: $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n - 1) \cdot n$. Der Sonderfall $0! = 1$ soll ebenfalls berücksichtigt werden. Zum Testen soll im Hauptprogramm die Fakultät von 0 bis 20 berechnet werden.

Programm:

```
#include <stdio.h>

unsigned int factorial(unsigned char n)
{
    int i;
    unsigned int retVal = 1;

    if (n == 0)
    {
        return 1;
    }

    for (i = 1; i <= n; i++)
    {
```



```

    retVal *= i;
}

return retVal;
}

void main(void)
{
    int i;

    for (i = 0; i <= 20; i++)
    {
        printf("%2d! = %10u\n", i, factorial(i));
    }
}

```

Hier wurde als Beispiel ein `unsigned char` als Übergabeparameter verwendet. Das soll nur zeigen, dass man mit `char` auch ganz normal rechnen kann. Außerdem wird die Fakultät relativ schnell sehr groß, sodass große Werte für n den Rückgabeparameter überlaufen lassen. Das erkennt man, wenn man das Programm laufen lässt. Die Ausgabe sieht so aus:

```

0! =      1
1! =      1
2! =      2
3! =      6
4! =     24
5! =    120
6! =    720
7! =   5040
8! =  40320
9! = 362880
10! = 3628800
11! = 39916800
12! = 479001600
13! = 1932053504
14! = 1278945280
15! = 2004310016
16! = 2004189184
17! = 4006445056
18! = 3396534272
19! = 109641728
20! = 2192834560

```

Die Berechnung der Fakultät dürfte demnach bei 16! bereits überlaufen (Maximum: 4 294 967 295). Es gibt noch einen längeren Datentyp als `int/long`, den wir bisher nicht erwähnt haben: `long long` (tatsächlich $2 \times$ `long` hintereinander geschrieben!). Dieser Datentyp ist 64 bit breit. Damit kann die Fakultät zumindest bis 21! berechnet werden. Die Ausgabe per `printf()` erfolgt über `%llu` (*long-long unsigned*).

Aufgabe 1: Binomialkoeffizienten

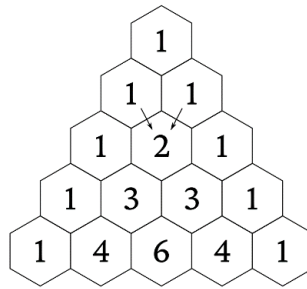
Schreibe eine Funktion zur Berechnung eines Binomialkoeffizienten. Die Binomialkoeffizienten können berechnet werden mit der Formel: $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Den linken Teil spricht man » n über k « aus. Damit kann man den binomischen Lehrsatz auf beliebige n anwenden:

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$$

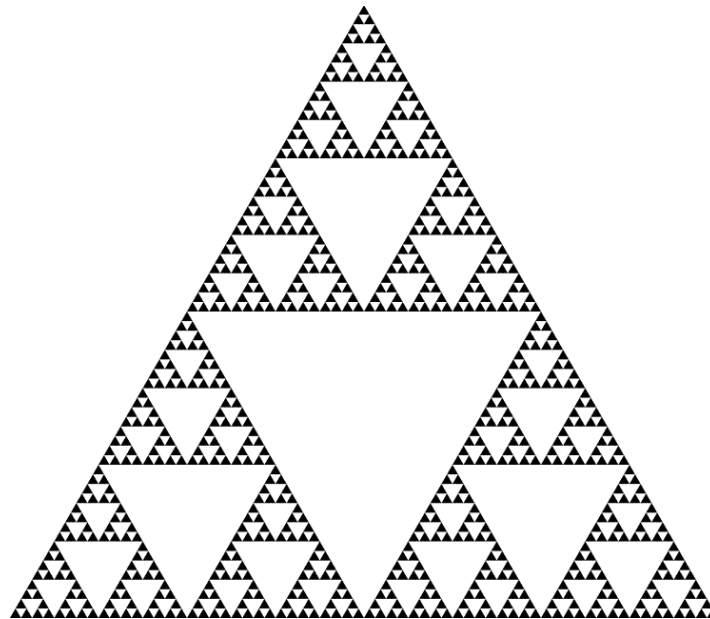
n und k sind positive ganze Zahlen. Normalerweise ist $k \leq n$. Für den Fall $k > n$ soll die Funktion 0 zurückgeben. Teste die Funktion mit $n = 4$ und lass k von 0 bis 4 durchlaufen. Das Ergebnis sollte 1, 4, 6, 4, 1 sein.

Aufgabe 2: Pascalsches Dreieck

Das Pascalsche Dreieck ist eine grafische Veranschaulichung der Binomialkoeffizienten. Man beginnt an den beiden Seiten jeweils mit 1. Die Felder innen ergeben sich immer als Summe der Felder links und rechts darüber.



Wenn man das Dreieck zeilenweise liest, dann erhält man genau die Binomialkoeffizienten aus der letzten Aufgabe. In der ersten Zeile ist somit $n = 0$, dann $n = 1$, usw. Ein interessantes Detail am Rand: malt man jeweils die ungeraden Felder eines Pascalschen Dreiecks schwarz an und lässt die geraden Felder weiß, so erhält man das Sierpinski-Dreieck, ein Fraktal:



Nun aber zur Aufgabe: schreibe ein Programm, dass das Pascalsche Dreieck bis $n = 12$ ausgibt. Versuch zuerst, das Dreieck als rechtwinkeliges Dreieck auszugeben, z. B. so:

```
n = 0:  1
n = 1:  1  1
n = 2:  1  2  1
n = 3:  1  3  3  1
n = 4:  1  4  6  4  1
n = 5:  1  5 10 10  5  1
n = 6:  1  6 15 20 15  6  1
n = 7:  1  7 21 35 35 21  7  1
n = 8:  1  8 28 56 70 56 28  8  1
```

usw. Wenn das klappt, dann versuche genauso viele Leerzeichen einzufügen, dass das Dreieck gleichschenkelig wird:

```
n = 0:           1
n = 1:         1  1
n = 2:       1  2  1
n = 3:     1  3  3  1
n = 4:   1  4  6  4  1
n = 5: 1  5 10 10  5  1
n = 6: 1  6 15 20 15  6  1
n = 7: 1  7 21 35 35 21  7  1
n = 8: 1  8 28 56 70 56 28  8  1
```

Zusatzaufgabe:

Unter Windows ist es sogar möglich, Text farblich auszugeben (siehe 4.2). Färbe alle ungeraden Koeffizienten rot ein und lass die geraden weiß.

- Diese Zeile einfügen ändert die Farbe des nachfolgenden Textes auf rot:

```
SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), 12);
```

- Diese Zeile einfügen ändert die Farbe des nachfolgenden Textes wieder auf weiß:

```
SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), 15);
```

Aufgabe 3: Eulersche Zahl

Die Eulersche Zahl $e = 2,718281828459045235\dots$ lässt sich als Summe der Kehrwerte der Fakultäten definieren:

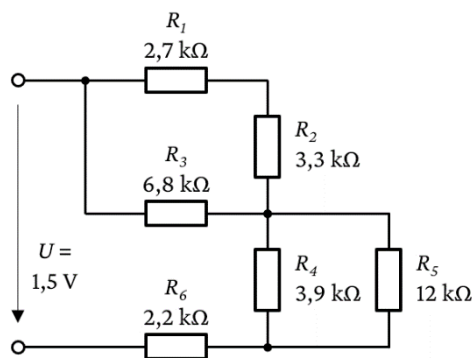
$$e = \sum_{k=0}^{\infty} \frac{1}{k!} = 1 + \frac{1}{1!} + \frac{1}{2!} + \dots$$

Schreib ein Programm, dass die e -Funktion näherungsweise mit dieser Formel berechnet. Verwende einen `double` für e . Lass k von 0 bis 12 durchlaufen und gib die Zwischensumme jeweils auf 10 Kommastellen genau aus. Ab wann stimmt das Ergebnis auf zumindest 5 Stellen genau? Vergleiche dazu mit der Konstanten `M_E` (Bibliothek `math.h`).

Aufgabe 4: Widerstandsberechnung

Erstelle eine Funktion zur Berechnung der Parallelschaltung und eine weitere Funktion zur Berechnung der Serienschaltung zweier Widerstände. Welcher Vorteil ergibt sich im Vergleich zum Widerstandsbeispiel aus dem letzten Kapitel?

Berechne den Gesamtwiderstand und den Gesamtstrom der nachfolgenden Schaltung mit Hilfe der beiden Funktionen.



12.4 Rekursive Funktionen

Rekursive Funktionen sind Funktionen, die sich selbst aufrufen. Manche Aufgaben können damit sehr elegant gelöst werden. Man muss jedoch aufpassen, dass sich rekursive Funktionen nicht endlos (oder zu oft) selbst aufrufen; sonst kommt es zu einem sogenannten *stack overflow*.

Beispiel 1: Fakultät mittels rekursiver Funktion

```
unsigned int factorial(unsigned int n)
{
    if ((n == 0) || (n == 1))
    {
        return 1;
    }
    else
    {
        return n * factorial(n - 1);
    }
}
```

Beispiel 2: größter gemeinsamer Teiler (ggT) zweier Zahlen

Rekursiv über den Algorithmus von Euklid:

- $\text{ggT}(x, y) = x + y$, falls $x = 0$ oder $y = 0$
- $\text{ggT}(x, y) = \text{ggT}(y, x \bmod y)$, sonst

Programm:

```
unsigned int ggt1(unsigned int x, unsigned int y)
{
    if ((x == 0) || (y == 0))
```

```

{
    return x + y;
}
else
{
    return ggt1(y, x % y);
}
}

```

Nichtrekursive Variante:

```

unsigned int ggt2(unsigned int x, unsigned int y)
{
    unsigned int temp;

    while (y)
    {
        temp = y;
        y = x % y;
        x = temp;
    }
    return x;
}

```

Aufgabe 1

Teste die beiden ggt-Funktionen mit den Zahlenpaaren 560, 91 und 972, 666. Ergebnis: 7 bzw. 18.

Aufgabe 2: Algorithmus von Nicomachos (100 n. Chr.)

Es gibt noch einen rekursiven Algorithmus zur Berechnung des ggt:

- $\text{ggt}(x, y) = 1$, falls $x = 1$ oder $y = 1$
- $\text{ggt}(x, y) = x$, falls $x = y$
- $\text{ggt}(x, y) = \text{ggt}(x - y, y)$, falls $x > y$
- $\text{ggt}(x, y) = \text{ggt}(x, y - x)$, sonst

Schreibe eine zusätzliche Funktion `ggt3()`, die den Algorithmus von Nicomachos implementiert. Verwende die Zahlen der letzten Aufgabe zur Überprüfung der Funktion.

Wir haben bereits eine Menge von Funktionen geschrieben, die einen Parameter zurückgeben. Manchmal wäre es auch praktisch, wenn zwei oder mehr Parameter zurückgegeben werden könnten. Leider ist das in C nicht so ohne weiteres möglich. Es geht aber über Umwege. Dafür müssen wir uns zuerst mit Zeigern vertraut machen.

13 Zeiger bzw. Pointer

Zeiger (engl. *pointer*) sind zu Beginn meist schwer zu verstehen. Wenn man aber das Prinzip dahinter verstanden hat, dann sind sie ein mächtiges Werkzeug bei der Programmierung in C/C++. Vor allem bei kleinen Prozessoren (Microcontroller) kann man damit sehr effizient programmieren.

Wie der Name *Zeiger* schon sagt, zeigt dieser irgendwohin. Gemeint ist die Speicheradresse, an der eine Variable gespeichert wird. Ganz zu Beginn bei den Variablen wurde schon erwähnt, dass Variablen immer auch eine Speicheradresse haben. Und damit wollen wir jetzt arbeiten.

Indirekt haben wir bereits mit Zeigern gearbeitet, nämlich bei der Verwendung von `scanf_s("%f", &var)`. Da haben wir immer ein `&` (Adressoperator) vor die Variable geschrieben. Damit kann man die Adresse einer Variable herausfinden.

Beispiel:

```
int x;
x = 10;

printf("Wert von x:    %d\n", x);
printf("Adresse von x: %p\n", &x);
```

Wenn man das Programm öfter laufen lässt, dann kommen verschiedene Adressen heraus, z. B. 003DF860 (hex). Das liegt daran, dass am Computer die Speicherbereiche dynamisch verwaltet werden. Beim `scanf_s()` wird die Adresse von `var` übergeben und nicht `var` selbst. Dadurch kennt die Funktion `scanf_s()` die Adresse der Variable und bekommt nicht einfach eine Kopie des Werts. An der Stelle der Adresse kann die Variable geändert werden. Somit kann `scanf_s()` die Variable `var` ändern, obwohl sie eigentlich nur ein Übergabeparameter ist.

Beim letzten Beispiel ist `&x` also ein Zeiger. Wir können aber auch direkt Zeiger-Variablen anlegen. Dann ist es umgekehrt: wir arbeiten mit der Adresse anstelle des Variablenwerts. Für den Zugriff auf den Wert brauchen wir einen Wert-Operator. Um Variablen als Zeiger-Variablen zu deklarieren schreibt man:

```
int* y;
float* z;
```

Die beiden Variablen sind eigentlich normale `int` bzw. `float`, aber wenn man mit `=` zuweist, dann ist ab jetzt die Adresse anstelle des Werts gemeint. Bei der Variablendeklaration eines Pointers ist noch nicht bekannt, welche Adresse dieser später haben wird; diese muss erst zugewiesen werden. Beispiel:

```
#include <stdio.h>

void main(void)
{
    int x;
    int* y;

    x = 10;    // x einen Wert zuweisen
    y = &x;    // y die Adresse von x zuweisen

    printf("Wert von x:    %d\n", x);
    printf("Adresse von x: %p\n", &x);

    printf("Wert von y:    %d\n", *y);
    printf("Adresse von y: %p\n", y);
}
```

Was passiert hier? `x` kennen wir bereits. `y` kann man einfach die Adresse von `x` zuweisen, indem man `&` davor schreibt. Weil `y` ja schon ein Zeiger ist, kann man die Adresse in der letzten Zeile direkt mit `y` ausgeben. Möchte man aber auf den Wert zugreifen, auf den `y` zeigt, dann benötigt man auch hier einen `*` (engl. *asterisk*). Das Beispiel zeigt, dass beide Ausgaben dasselbe Ergebnis liefern, weil `y` nichts anderes macht als auf `x` zu zeigen.

Sobald der Zeiger `y` eine Adresse bekommen hat, kann man ihm ganz normal Werte zuweisen:

```
*y = 20;    // *y einen Wert zuweisen

printf("Wert von y:    %d\n", *y);
printf("Adresse von y: %p\n", y);

printf("Wert von x:    %d\n", x);
printf("Adresse von x: %p\n", &x);
```

Da y auf x zeigt, ist die Ausgabe bei beiden mit dem Wert 20 wieder dieselbe.

Noch einmal zum Vergleich:

	»normale« Variable int x	Zeiger-Variable int* y
Wert zuweisen	x = 10;	*y = 10;
Wert ausgeben	printf("%d\n", x);	printf("%d\n", *y);
Adresse ausgeben	printf("%p\n", &x);	printf("%p\n", y);

Bei den Operatoren tut man sich vielleicht leichter, wenn man sich angewöhnt, dass man zu *y sagt: »Wert von y« und zu &x sagt: »Adresse von x«.

Aber wozu das alles? Wofür braucht man Zeiger? Damit können z. B. Funktionen geschrieben werden, bei denen mehr als 1 Wert zurückgeliefert bzw. verändert wird. Oder es können große Mengen an Daten übergeben werden, ohne dass alles kopiert werden muss. Schauen wir uns zunächst die Funktionen an.

13.1 Funktionen mit Zeigern

Als einfaches Beispiel wollen wir einem Unterprogramm den Radius eines Kreis übergeben und es soll uns den Umfang und die Fläche berechnen. Wir wollen keine globalen Variablen dafür verwenden.

Programm:

```
#define _USE_MATH_DEFINES
#include <math.h>
#include <stdio.h>

void circle(float radius, float* circum, float* area)
{
    *circum = 2 * M_PI * radius;    // Wert von Umfang = ...
    *area = radius * radius * M_PI; // Wert von Fläche = ...
}

void main(void)
{
    float r, c, a;
    r = 10.0;
    circle(r, &c, &a);    // Aufruf mit 1x float, 2x Adresse von float
    printf("Umfang: %.2f, Fläche: %.2f", c, a);
}
```

Was passiert hier?

1. Die Übergabeparameter, die durch das Unterprogramm verändert werden sollen, sind als Zeiger zu deklarieren: float* circum, float* area
2. Im Unterprogramm werden die Werte der Variablen, auf die die Zeiger circum bzw. area zeigen, entsprechend geändert
3. Beim Aufruf von circle() wird der Radius ganz normal übergeben (als Kopie, engl. *by value*), während die zu ändernden Variablen als Adresse (engl. *by address*) übergeben werden
4. Das Unterprogramm hat also die Möglichkeit, c und a zu verändern, weil es deren Adresse kennt (»ich weiß, wo du wohnst«)

13.2 Unterschiede zwischen Ein- und Ausgangsparametern

Bei Eingangsparametern (Wertparameter) wird der Wert des Eingangsparameters auf eine lokale Variable des Unterprogramms übergeben (es wird dort eine Kopie erstellt, neuer Speicherplatz wird belegt). Diese Art des Aufrufs nennt man **call by value**. Nach Verlassen des Unterprogramms geht auch die lokale Variable bzw. Kopie verloren.

Bei einem Ausgangsparameter soll jedoch eine berechnete Größe (z. B. die berechnete Leistung eines Motors) auch nach Verlassen des Unterprogramms noch existieren. Mit globalen Variablen ist dies natürlich möglich, es entspricht aber nicht dem modernen Programmierstil.

Abhilfe: Es wird die Adresse einer Variablen des rufenden Programmteils übergeben. Diese Art des Aufrufs nennt man **call by reference**.⁴ Der übergebene Parameter heißt dann **Referenzparameter**. Das Unterprogramm verändert also den Wert der Variablen des rufenden Programmteils (main oder ein anderes Unterprogramm), kurz gesagt der Originalspeicherplatz wird verwendet.

Regel für Ausgangsparameter in C

- Im **gerufenen** Unterprogramm wird in der Parameterliste beim Datentyp ein * angehängt, z. B.:
`void rechteck (float a, float b, float* flaeche, float* umfang) usw.`
- Im **aufrufenden** Unterprogramm wird der Referenzparameter angegeben, z. B.: `rechteck(a, b, &fl, &umf);`

13.3 Aufgaben zu Funktionen mit Zeigern

Aufgabe 1: Rechteck

Schreibe ein Programm, dass im Hauptprogramm die Seiten a und b eines Rechtecks einliest. Berechne mit Hilfe eines Unterprogramms `void rectangle(...)` mit Zeigern (ohne globale Variablen) den Umfang und die Fläche des Rechtecks. Gib das Ergebnis im Hauptprogramm aus.

Erweiterung:

- Verwende ein weiteres Unterprogramm `void readParameter(...)` zum Einlesen der Parameter a und b
- Schreibe noch ein Unterprogramm `void writeResult(...)` zum Ausgeben der Rechenergebnisse

Aufgabe 2: Quader

Nimm das Rechteck-Programm als Ausgangsbasis und erweitere es auf einen Quader:

- Erweitern Einlesen auf a , b und c
- Ändern der Berechnung auf `void cuboid(...)` mit Oberfläche und Volumen
- Ändern der Ausgabe

Erweiterungen:

- Berechnung der Raumdiagonalen mit normalem Rückgabeparameter (ohne Zeiger)
- Zusätzliches Unterprogramm `void swap(...)`, welches die Seiten a und b vertauscht, wenn b größer als a ist

Aufgabe 3: Sekundenumwandler

Schreibe ein Unterprogramm, dass eine gegebene Anzahl von Sekunden in Stunden, Minuten und Sekunden umwandelt.

Aufgabe 4: Ganzzahldivision

Schreibe ein Unterprogramm, dass aus zwei INT-Eingangsgrößen den ganzzahligen Quotienten und den Divisionsrest errechnet.

Aufgabe 5: Temperaturumwandler

Schreibe ein Programm, dass aus einer gegebenen Temperatur in Grad Celsius die Temperaturen auf: Kelvin, Grad Fahrenheit, Grad Rankine, Grad Delisle, Grad Réaumur, Grad Newton und Grad Rømer umrechnet und in einem eigenen UP ausgibt. Für die Berechnung siehe Wikipedia.

Erweiterung 1: Erstelle für jeden Schritt (Eingabe, Umwandlung, Ausgabe) ein eigenes UP

Erweiterung 2: Erweitere das Programm, sodass ein Bereich $[T_{min}, T_{max}]$ eingegeben werden kann und eine Temperatur-tabelle ausgegeben wird.

⁴ Streng genommen heißt diese Art des Aufrufs in C **call by address**. In Programmiersprachen wie Java, Python o. ä. ist jedoch die Bezeichnung **call by reference** üblich, weshalb sie hier verwendet wird. In C++ gibt es noch weitere feine Unterschiede.

Aufgabe 6: Dreiersortierung

Bestimme im Hauptprogramm 3 Zufallszahlen z_1 , z_2 und z_3 und gib diese am Bildschirm aus. Diese Zahlen sollen an ein Unterprogramm `void sort(...)` übergeben werden. Dieses Unterprogramm soll die Zahlen derart sortieren, dass z_1 die kleinste, z_2 die mittlere und z_3 die größte Zahl zurückgibt. Gib die sortierten Zahlen im Hauptprogramm aus.

14 Felder bzw. Arrays

Felder (engl. *arrays*) setzen sich aus (mehreren) Elementen eines Datentyps zusammen. Als Beispiel ist ein `string` nichts anderes als ein Feld von `char`-Variablen.

14.1 Eindimensionales Feld (Vektor)

Ein 1-dimensionales Feld wird auch *Vektor* genannt. Für die Deklaration von Feldern verwendet man eckige Klammern `[]` und gibt darin die gewünschte Anzahl von Einträgen an.

Beispiel:

```
int varName[10]; // Definition eines Felds mit 10x int
```

Beispiele:

```
#define NUM_OF_VALUES 100

int   cards[4];           // Feld mit 4x int
float values[NUM_OF_VALUES]; // Feld mit 100x float
double valuesD[30];       // Feld mit 30x double
char  text[8];            // Feld mit 8x char
```

Der Compiler erkennt anhand der Klammern, dass es sich um ein Feld handelt. Feldelemente liegen ab einer (für den Programmierer zufälligen) Adresse beginnend lückenlos hintereinander im Speicher. Bei der Deklaration von `text` werden also z. B. 8 Bytes im Hauptspeicher reserviert; oder bei `cards` 4×4 Bytes = 16 Bytes. Nach der Deklaration von Feldern kann deren Größe nicht mehr verändert werden.

Der Zugriff auf ein einzelnes Feldelement erfolgt ebenfalls per `[]` durch Angabe eines *Index*. Dieser Index beginnt immer bei 0. Dadurch hat das letzte Element eines Feldes mit n Elementen den Index $n-1$. Ein Index muss natürlich immer positiv ganzzahlig sein; er kann auch das Ergebnis eines Ausdrucks sein.

Hier muss man sehr vorsichtig sein: eine Bereichsüberprüfung erfolgt nicht! Der Programmierer muss darauf achten, dass die Feldgrenzen nicht überschritten werden.

Besonders gefährlich sind Schreibzugriffe, die über die Feldgrenze hinausgehen; es könnte vorkommen, dass in Bereiche geschrieben wird, die vom Programm für andere Variablen belegt werden. Bei der Ausführung dieses Teils kommt es dann oft zu einem Absturz. Der Fehler wird dann zunächst in diesem Programmteil gesucht, obwohl die Ursache ganz woanders liegt.

Beispiele für den Zugriff auf Feldelemente:

```
int cnt, i;
cnt = 2;

valuesD[0] = 22.0;
valuesD[++cnt] = 13; // ++cnt erhöht zuerst auf 3,
                    // dann Zugriff auf valuesD[3]

cnt = 4;

valuesD[cnt++] = 15; // cnt++ greift zuerst auf valuesD[4] zu und
                    // erhöht danach auf 5

text[0] = 'a';

for (i = 0; i < NUM_OF_VALUES; i++)
{
    values[i] = 0.0; // alle Werte per Schleife auf 0.0 setzen
}
```

Beispiel:

5 Messwerte sollen als `float` gespeichert werden: $a_1 = 5.7$, $a_2 = 3.6$, ..., $a_5 = 28.6$.

Deklaration Variante 1:

```
float vals1[5];
```

Deklaration Variante 2:

```
const int NUM_OF_VALUES = 5;
```

```
float vals2[NUM_OF_VALUES];
```

Deklaration Variante 3 (bevorzugt):

```
#define NUM_OF_VALUES 5  
float vals3[NUM_OF_VALUES];
```

Anschließend kann auf das Feld zugegriffen werden. Beachte, dass der Wert a_1 in Index 0 landet, a_2 in Index 1, ..., a_5 in Index 4.

Initialisieren von Werten

Den Feldelementen kann man bereits bei der Deklaration Werte zuweisen. Dafür werden geschwungene Klammern {} verwendet.

Beispiel:

```
float measData[5] = { 1.2, 3.4, 5.6, 7.8, 9.0 };
```

Wenn Deklaration und Initialisierung zusammenfallen, dann kann auch die Größenangabe des Arrays entfallen:

```
float measData[] = { 1.2, 3.4, 5.6, 7.8, 9.0 };
```

In diesem Fall weiß der Compiler auch so, dass 5 Elemente benötigt werden. Es ist auch möglich, die Array-Größe anzugeben, aber nicht allen Elementen einen Wert zuweisen:

```
float measData[5] = { 1.2, 3.4 };
```

In diesem Fall werden alle nicht angegebenen Elemente mit 0 initialisiert. Um allen Elemente eines Feldes den Wert 0 zuzuweisen, verwendet man daher am besten:

```
float measData[5] = { 0 };
```

Wird nichts angegeben, also keine Werte in {}, so haben die Elemente je nach Compiler entweder keine eindeutig definierten Werte oder werden ebenfalls alle mit 0 initialisiert.

Eingabe der Elemente zur Laufzeit

```
#include <stdio.h>  
#define NUM_OF_VALUES 4  
  
void main(void)  
{  
    int i;  
    int measData[NUM_OF_VALUES] = { 0 };  
  
    for (i = 0; i < NUM_OF_VALUES; i++)  
    {  
        printf("Bitte %d. Zahl eingeben:\n", i + 1);  
        scanf_s("%d", &measData[i]);  
    }  
}
```

Beachte auch hier die Verwendung des Adress-Operators & beim Aufruf von `scanf_s()`! Hier kommt zusätzlich noch der Index des Feldes hinzu.

Ausgabe von Elementen

```
for (i = 0; i < NUM_OF_VALUES; i++)  
{  
    printf("%d\n", measData[i]);  
}
```

14.2 Mehrdimensionales Feld (Matrix)

Es ist auch möglich, mehrdimensionale Felder zu erstellen. Diese werden auch *Matrix* (pl. *Matrizen*) genannt.

Beispiel für ein 2-dimensionales Feld: Schachbrett

```
#define NUM_OF_ROWS    8    /* Anzahl der Reihen */
#define NUM_OF_COLUMNS  8    /* Anzahl der Spalten */

int chessBoard[NUM_OF_ROWS][NUM_OF_COLUMNS];
```

Aufgabe: Schachbrett initialisieren

Verwende die Deklaration des Schachbretts oben und initialisiere alle Werte mit 0. Verschachtelte for-Schleifen sind dafür gut geeignet. Gibt nach der vollständigen Initialisierung alle Werte am Bildschirm aus, um zu prüfen, ob die Initialisierung richtig funktioniert.

Man kann auch mehrdimensionale Arrays gleich bei der Deklaration initialisieren. Dazu werden die geschwungenen Klammern verschachtelt.

Beispiel:

```
int matrix[2][3] = { { 1, 2, 3 }, {4, 5, 6 } };
```

Es sind prinzipiell auch mehr als zwei Dimensionen möglich. Das wird aber relativ selten gebraucht.

14.3 Felder als Übergabeparameter

Felder können auch an Unterprogramme übergeben werden.

Beispiel: so geht es prinzipiell, es ist aber nicht optimal ...

```
#include <stdio.h>
#define NUM_OF_VALUES 10

void printData(int vals[])
{
    int i;
    for (i = 0; i < NUM_OF_VALUES; i++)
    {
        printf("%d\n", vals[i]);
    }
}

void main(void)
{
    int values[NUM_OF_VALUES] = { 0 };

    printData(values);
}
```

Das Beispiel ist nicht gut, weil das Unterprogramm wieder auf einen globalen Parameter (NUM_OF_VALUES) angewiesen ist. Sonst weiß das Unterprogramm nicht, wie viele Elemente das Feld besitzt (auch nicht per sizeof()). Es ist daher üblich, dass man als zusätzlichen Übergabeparameter die Länge des Feldes mitgibt.

Beispiel: so ist es besser ...

```
#include <stdio.h>
#define NUM_OF_VALUES 10

void printData(int vals[], int len)
{
    int i;
    for (i = 0; i < len; i++)
    {
        printf("%d\n", vals[i]);
    }
}
```

```
void main(void)
{
    int values[NUM_OF_VALUES] = { 0 };

    printData(values, NUM_OF_VALUES);
}
```

Beachte, dass beim Aufruf des Unterprogramm keine [] angegeben werden! Sobald ein Feld als Parameter verwendet wird, wird dieser Parameter (also das Feld) nicht wie ein normaler Übergabeparameter kopiert (*by value*), sondern es wird eigentlich nur die Adresse übergeben (*by address*). In dieser Hinsicht verhält sich ein Feld genau gleich wie ein Zeiger. Der Aufruf von `printData()` mit `values` könnte auch erfolgen, indem man `&values[0]` verwendet. Es wird also die Adresse des 1. Elements übergeben. Dadurch, dass das Feld hintereinander im Speicher liegt, kann einfach auf alle anderen Elemente zugegriffen werden.

Die Übergabe eines Felds funktioniert im Prinzip auch mit mehrdimensionalen Feldern.

14.4 Aufgaben zu Feldern

Aufgabe 1: Messwerte

1. Erzeuge ein Feld mit 10 Elementen vom Typ `float`
2. Initialisiere das Feld mit beliebigen Werten
3. Gib das Feld am Bildschirm aus
4. Addiere einen fixen Wert zu jedem Element
5. Finde das Maximum aller Werte
6. Finde das Minimum aller Werte
7. Berechne die Summe und den Mittelwert über alle Elemente
8. Überschreibe das Feld durch manuelle Eingabe
9. Kopiere das Feld in ein zweites Feld. Bonus: kopiere es in umgekehrter Reihenfolge!

Aufgabe 2: Erweiterung von Aufgabe 1

Erweitere das Programm zur Erfassung und Auswertung von Messwerten durch folgenden Unterprogrammen:

```
/* Erzeugen von Messdaten mittels Zufallsgenerator */
void generateData(float data[], int len) { /* ... */ }

/* Manuelle Eingabe von Messdaten */
void readData(float data[], int len) { /* ... */ }

/* Anzeige der Daten */
void printData(float data[], int len) { /* ... */ }

/* Rückgabe des Maximalwertes */
float maximum(float data[], int len) { /* ... */ }

/* Rückgabe des Minimalwertes */
float minimum(float data[], int len) { /* ... */ }

/* Berechnung des Mittelwerts */
float average(float data[], int len) { /* ... */ }

/* Berechnung der Standardabweichung */
float stddev(float data[], int len) { /* ... */ }
```

Verwende für die Standardabweichung folgende Formel:

$$s = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N - 1}}$$

wobei x_i die einzelnen Elemente sind, N die Anzahl der Elemente und $\bar{x}$ der Mittelwert. Eventuell mit der Excel-Funktion `=STABW.S()` testen.

Aufgabe 3: Erweiterung von Aufgabe 2

Erweitere das vorherige Programm noch einmal um zwei Funktionen. Erstens:

```
/* Vertausche die Elemente mit index1 und index2 im Array data */
void swap(float data[], int len, int index1, int index2) { /* ... */ }
```

Prüfe dabei auch, ob die beiden Indizes auch innerhalb der Array-Grenzen liegen.

Sortiere zweitens alle Elemente des Feldes mit Hilfe des *Bubble-Sort* Algorithmus:

```
/* Sortieren der Messwerte */
void bubbleSort(float data[], int len) { /* ... */ }
```

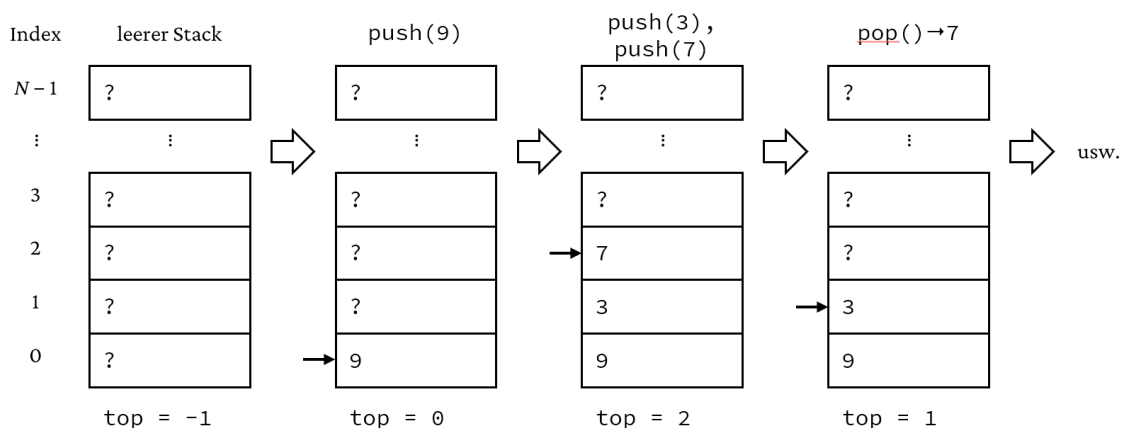
Infos zur Implementierung von Bubble-Sort siehe Internet.

Aufgabe 4: Stack

Bei dieser Aufgabe soll ein *Stack* (von engl. Stapel) mit einer Größe von 10 Elementen mittels float-Array implementiert werden. Ein Stack ist ein sogenannter *LIFO*-Speicher (engl. *last in, first out*). Man kann sich das wie einen Berg aus T-Shirts vorstellen. Kommt ein neues T-Shirt in den Kleiderschrank, dann landet es oben auf dem Stapel. Wird ein T-Shirt benötigt, dann kommt das oberste zuerst weg.⁵

Das Ablegen eines Wertes auf dem Stack nennt man »to *push*«. Das Herunternehmen eines Wertes vom Stack heißt »to *pop*«. Mit *top* wird oft der Index des obersten Elements bezeichnet. Wie in C üblich ist es sinnvoll, den 1. Eintrag des Stacks mit dem Index 0 zu assoziieren; ein leerer Stack hat dann einen *top*-Wert von -1. Die Abbildung soll die Funktion veranschaulichen:

- Zuerst ist der Stack leer
- Dann wird 9 gepushed
- Anschließend werden noch 3 und dann 7 gepushed
- Als nächstes wird per *pop* der oberste Eintrag herunter genommen



Benutze zur Implementierung des Stacks folgende Funktionen:

```
/* Initialisieren des Stacks mit Werten 0.0 */
void initStack(float stack[], int len) { /* ... */ }

/* Alle Werte des Stacks ausgeben */
void printStack(float stack[], int top) { /* ... */ }

/* Zahl auf dem Stack ablegen */
void pushStack(float stack[], int len, float value, int* top) { }

/* Zahl vom Stack nehmen */
float popStack(float stack[], int* top) { /* ... */ }
```

⁵ Beim Kleiderschrank wäre ein FIFO-Speicher (*first in, first out*) besser geeignet, damit sich die Kleidung möglichst abwechselt bzw. gleich oft getragen wird.

Folgendes soll berücksichtigt werden:

- `printStack()` wenn `top=-1` ist, soll eine Meldung erfolgen, dass der Stack leer ist
- `pushStack()` wenn der Stack bereits voll ist, soll die neue Variable ignoriert werden und eine Meldung »Stack overflow!« erfolgen; wenn der Stack nicht voll ist, soll ausgegeben werden, an welcher Position der Wert abgelegt wurde
- `popStack()` wenn der Stack bereits leer ist, soll 0.0 zurückgegeben werden und eine Meldung erfolgen, dass der Stack leer ist; wenn der Stack nicht leer ist, soll der Wert und die Position des Werts ausgegeben werden

Achte darauf, dass sämtliche Funktionen ohne globale Variablen implementiert werden! Überlege auch gut, welche Übergabeparameter als Pointer deklariert sind und warum.

Zum Testen des Stacks mache im Hauptprogramm folgendes:

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE_OF_STACK 10

// Unterprogramme komme hier her

void main(void)
{
    float stack[SIZE_OF_STACK];
    float rnd;
    int top = -1;
    int i;

    initStack(stack, SIZE_OF_STACK);
    printStack(stack, top);

    // 5 Werte ablegen
    for (i = 0; i < 5; i++)
    {
        rnd = (float) rand() / RAND_MAX;
        pushStack(stack, SIZE_OF_STACK, rnd, &top);
    }

    printStack(stack, top);

    // 3 Werte abholen
    for (i = 0; i < 3; i++)
    {
        popStack(stack, &top);
    }

    printStack(stack, top);

    // 9 Werte ablegen -> overflow beim 9. Wert
    for (i = 0; i < 9; i++)
    {
        rnd = (float) rand() / RAND_MAX;
        pushStack(stack, SIZE_OF_STACK, rnd, &top);
    }

    printStack(stack, top);

    // 11 Werte abholen -> leer beim 11. Wert
    for (i = 0; i < 11; i++)
    {
        popStack(stack, &top);
    }

    printStack(stack, top);
}
```

Hinweis: auch C verwendet im Hintergrund einen Stack. Dort werden z. B. Übergabeparameter abgelegt oder auch Adressen vom Programm (sind ja auch nur Werte im Speicher). Damit findet das Programm später aus einem Unterprogramm wieder ins Hauptprogramm zurück.

Aufgabe 5: Erweiterung von Jäger und Wolf

Bei jedem Fangen eines Wolfs wird der Jäger um 1 Zeichen länger und bewegt sich dabei wie ein Wurm über das Spielfeld. Fährt der Jäger in seine eigene Spur, dann hat er verloren. Dieses Spiel gab's auf alten Nokia-Handys unter dem Namen »snake«.

Aufgabe 6: Sieb des Eratosthenes

Ermittle die Primzahlen bis 200 mit Hilfe des Siebs des Eratosthenes. Dabei werden schrittweise die Vielfachen der Zahlen beginnend bei 2 gestrichen (ersetzen durch -1). Hier die ersten Durchgänge:

Start:	1	2	3	4	5	6	7	8	9	10	11	...
2er-Reihe:	1	2	3	-1	5	-1	7	-1	9	-1	11	...
3er-Reihe:	1	2	3	-1	5	-1	7	-1	-1	-1	11	...

Am Ende bleiben die Primzahlen über. Überlege dir, welche Optimierungsmöglichkeiten es gibt!

Aufgabe 7: Zahlenfolge aufteilen

Erstelle eine zufällige Zahlenfolge mit 20 Elementen mit Zahlen von 1 bis 100. Teile diese Folge auf je eine Folge mit allen geraden und eine Folge mit allen ungeraden Zahlen auf. Beispiel:

Folge:	10	22	33	41	16	73	81	...
Gerade:	10	22	16	...				
Ungerade:	33	41	73	81	...			

Aufgabe 8: monotone Zahlenfolgen (+)

Es sollen zwei monoton steigende Zahlenfolgen mit je 10 Werten zu einer monoton steigenden Zahlenfolge zusammengefasst werden. Doppelte Einträge werden beim Zusammenführen gelöscht. Beispiel

Folge 1:	1	2	4	6	7	...		
Folge 2:	3	4	7	8	...			
Zusammen:	1	2	3	4	6	7	8	...

Aufgabe 9: Zeiger auf Feld

Befülle ein `int`-Feld zahlen mit 100 Zahlen der 2er Reihe. Deklariere einen Zeiger, der auf das erste Element zeigt und gehe das Feld mit Hilfe des Zeigers durch und gib die Elemente aus.

15 Strings (Zeichenketten)

15.1 Strings sind Felder

Eine Zeichenkette ist nichts anderes als ein char-Array, welches durch eine 0 (Wert 0, nicht ASCII-Zeichen '0', sondern Sonderzeichen '\0') abgeschlossen wird. Das Ende des strings wird also durch ein besonderes Zeichen markiert. Auf Englisch sagt man auch *zero-terminated string* bzw. *null-terminated string* dazu.

15.2 Deklaration von Zeichenketten

Bei der Deklaration muss dafür gesorgt werden, dass das 0-Zeichen ('\0') auch noch Platz hat.

```
char vorname[81];      /* maximale Länge : 80 Zeichen */
char nachname[301];    /* maximale Länge : 300 Zeichen */
char test[] = "Hallo!"; /* Es werden 7 Zeichen reserviert */

printf("%d", sizeof(test)); // gibt 7 aus
```

Jetzt erscheint auch das Beispiel aus Abschnitt 4.3 sinnvoll:

```
char text[11];          /* string aus 10 Zeichen + 0 am Ende */
scanf_s("%10s", text, 11); /* strings mit max. 10 Zeichen */
```

Hinweis: statt 11 in der letzten Zeile bietet sich auch hier sizeof(text) an.

Achtung: kommt beim Text ein Leerzeichen vor, dann bricht scanf_s() dort ab (bzw. sonst nach Eingabe von *Enter*)! Möchte man ganze Texte inklusive Leerzeichen einlesen, dann muss die Funktion gets_s() (ebenfalls aus stdio.h, siehe auch weiter unten) verwendet werden.

15.3 Arbeiten mit Zeichenketten

Kopieren

Zeichenketten können nicht direkt mit Zuweisungen kopiert werden (die Anweisung text1 = text ist in C unzulässig). Zum Kopieren von Zeichenketten muss die Funktion strcpy_s() (Bibliothek string.h) verwendet werden:

```
char vorname[81];
char nachname[301];
char bla[10];
char test[] = "Huber";

strcpy_s(nachname, test); // 1. Parameter = Ziel, 2. Param. = Quelle
strcpy_s(vorname, "Franz"); // 2. Parameter als konstanter string
strcpy_s(bla, ""); // nur 0-Zeichen

printf("%s %s", vorname, nachname);
```

Achtung: wird strcpy_s() in einem Unterprogramm aufgerufen, so muss zusätzlich die Länge des strings mit übergeben werden:

```
strcpy_s(nachname, sizeof(nachname), test);
// 1. Parameter = Ziel, 2. Param. = Länge, 3. Param. = Quelle
```

Auf die einzelnen Elemente des strings – die characters – kann wie beim Array üblich zugegriffen werden:

```
vorname[2] = 'i'; // aus Franz wird Frinz
vorname[3] = 't'; // aus Frinz wird Fritz

printf("%s %s", vorname, nachname);
```

Vergleichen

Zum Vergleichen dient die Funktion strcmp(). Verglichen werden die ASCII-Codes vom erstem und zweiten String. Mögliche Resultate:

1. 0 bei Gleichheit,
2. Wert >0, wenn der erste Parameter alphabetisch vor dem zweiten kommt
3. Wert <0, wenn der erste Parameter alphabetisch nach dem zweiten kommt

Beispiele:

```
printf("%d\n", strcmp("MAUS", "LAUS")); /* Ergebnis : 1 */
printf("%d\n", strcmp("HAI", "HAIFISCH")); /* Ergebnis : -1 */
printf("%d\n", strcmp("HAUS", "LAUS")); /* Ergebnis : -1 */
printf("%d\n", strcmp("Maus", "MAUSI")); /* Ergebnis : 1 */
```

Soll zwischen Groß- und Kleinschreibung nicht unterschieden werden, ist die Funktion `_stricmp()` zu verwenden. Aufgrund des Rückgabeparameters bietet sich an, folgende `if`-Anweisung zu verwenden:

```
if (!strcmp("MAUS", "LAUS"))
{
    printf("strings sind ident\n");
}
else
{
    printf("strings unterscheiden sich\n");
}
```

15.4 Sonstige Bibliotheksfunktionen für Strings und Characters

C stellt eine große Anzahl von Funktionen zur String-Bearbeitung mit der Bibliothek `string.h` zur Verfügung. Einige davon sind:

Funktion	Was macht die Funktion?
<code>strcat_s(str1, str2);</code>	Hängt <code>str2</code> an <code>str1</code> an; <code>str1</code> muss groß genug sein
<code>strlen(str);</code>	Gibt die Länge des strings zurück, also bis 0
<code>strstr(str1, str2);</code>	Sucht <code>str2</code> als Teilstring in <code>str1</code> ; wird er gefunden, wird der string ab dieser Position zurückgegeben, sonst 0 bzw. ein leerer string
<code>strchr(str1, ch);</code>	Gleich wie vorherige Funktion, jedoch mit nur einem Zeichen
<code>gets_s(str, len);</code>	liest einen String von der Tastatur; die Eingabe wird mit »Enter« (= NewLine = <code>\n</code>) beendet. »Enter« wird im String durch ein 0-Zeichen ersetzt. Für die Länge verwendet man am besten <code>sizeof(str)</code> . ⁶
<code>puts(str);</code>	gibt den String <code>str</code> am Bildschirm aus, das 0-Zeichen am Ende wird durch <code>\n</code> ersetzt.

Weiters gibt es in der Bibliothek `stdlib.h` einige praktische Funktionen zum Einlesen und Ausgeben von strings.

Beispiel zum Schreiben einer Zahl in einen string:

```
char str[6];
sprintf_s(str, "%5.3f", 1.234);
```

Beispiel zum Umwandeln einer Zahl als string in eine Zahl:

```
char str[] = "1.23456";
double val;
sscanf_s(str, "%lf", &val);
```

15.5 Übergabe von Strings an Unterprogramme

Da es sich bei Zeichenketten um Arrays handelt, gelten für die Übergabe an Unterprogramme dieselben Regeln wie für die Übergabe von Feldern. Beispiel:

```
#include <stdio.h>

void printSomething(char str1[])
{
    printf("%s\n", str1);
}
```

⁶ Es geht auch ohne Angabe der Länge. Aber erstaunlicherweise nur in Hauptprogrammen!

```

}

void main(void)
{
    char str[] = "Hello World!";
    printSomething(str);
}

```

Achtung: dieses Programm funktioniert nur, wenn man einen string als Variable übergibt. Möchte man einen konstanten string übergeben, so führt das zu einem Fehler. Geht nicht:

```

#include <stdio.h>

void printSomething(char str1[])
{
    printf("%s\n", str1);
}

void main(void)
{
    printSomething("Hello World!"); // string als Konstante
}

```

Das geht deshalb nicht, weil es ja sein könnte, dass das Unterprogramm das Array verändert. Um sicher zu gehen, dass das nicht passiert, muss man den Übergabeparameter als konstant deklarieren. Geht:

```

#include <stdio.h>

void printSomething(const char str1[])
{
    printf("%s\n", str1);
}

void main(void)
{
    printSomething("Hello World!");
}

```

Beachte, dass bei strings keine Angabe der Länge notwendig ist. Es wird solange zugegriffen, bis 0 erreicht wird. Bei allgemeinen Arrays kann man davon nicht ausgehen.

15.6 Aufgaben zu Strings

Aufgabe 1: vollständiger Name

- Erstelle 3 strings mit dem Namen anrede, vorname und nachname
- Initialisiere die 3 strings mit deinen Daten
- Kopiere die 3 strings auf einen gemeinsamen string name zusammen; füge Leerzeichen zwischen die einzelnen Teil ein
- Gib die variable name aus

Aufgabe 2: String gespiegelt ausgeben

- Lese einen string über den Bildschirm ein
- Spiegle den string: aus "Hallo" soll "ollaH" werden
- Gib den gespiegelten string aus

Aufgabe 3: Palindrom-Check

Erweitere die vorherige Aufgabe, indem du überprüfst, ob es sich bei dem string um ein Palindrom handelt.

Aufgabe 4: Buchstabenzähler

Schreib ein Programm, das einen string einliest. Gib aus, wie oft jeder einzelne Buchstabe im string vorkommt.

Beispiel:

Eingabe: „Hallo Welt“

Ausgabe:

```
H 1
a 1
l 3
o 1
W 1
e 1
t 1
```

Option: sortiere die Ausgabe alphabetisch.

Aufgabe 5: b-Sprache

Schreibe ein Programm, dass einen string einliest. Ersetze alle Vokale des strings durch Vokal + 'b' + Vokal.

Beispiel: aus „Hallo“ wird „Habalobo“

Aufgabe 6: String-Prozessor

Kombiniere die Aufgaben »string gespiegelt ausgeben«, »Buchstabenzähler« und »b-Sprache« zu einem gemeinsamen Programm. Verwende folgende Unterprogramme:

```
/* Einlesen eines strings */
void readString(char str[]) { /* ... */ }

/* Ausgabe des strings */
void printString(char str[]) { /* ... */ }

/* string spiegeln */
void reverseString(char str[]) { /* ... */ }

/* Zeichen zählen */
void countChar(char str[]) { /* ... */ }

/* b-Sprache: Eingabe in str, Ausgabe in strB */
void bLanguage(char str[], char strB[]) { /* ... */ }
```

Teste alle Funktionen mit einem geeigneten Hauptprogramm.

Aufgabe 7: Zeichenzähler

- Lese einen Satz als string ein
- Lese zwei einzelne Zeichen als char-Variablen ein
- Schreibe ein Unterprogramm, welches den Satz und die 2 Zeichen übergeben bekommt und feststellt, wie oft die beiden Zeichen im Satz vorkommen
- Gib das Ergebnis am Bildschirm aus

16 Enumeration (Aufzählungen)

Mit *enumerations* (Schlüsselwort *enum*) kann man einfache Aufzählungen in Form von `int`-Werten erstellen.

Beispiel:

```
enum weekDays
{
    Monday,      /* Wert 0 */
    Tuesday,     /* Wert 1 */
    Wednesday,   /* Wert 2 */
    Thursday,    /* ... */
    Friday,
    Saturday,
    Sunday       /* kein , beim letzten Eintrag */
};
```

Wie in C auch woanders üblich beginnt die Nummerierung bei 0.

Verwendung:

```
enum weekDays today;
today = Wednesday;
printf("Tag: %d\n", today);
```

Es können auch individuelle Werte vergeben werden, z. B.:

```
enum oddMonths { January = 1, March = 3, May = 5, July = 7 };
```

Alle Werte dürfen dabei nur 1 Mal vorkommen. Es ist auch möglich, Synonyme für enums zu verwenden. Dadurch entfällt bei der Deklaration das `enum` und die Aufzählung kann wie ein neuer Datentyp benutzt werden. Für Synonyme dient das Schlüsselwort `typedef` vor `enum`. Als Beispiel eine eigene Definition für eine Boolesche Variable:

```
typedef enum { False, True } bool_e;
bool_e isEmpty = True;
```

In diesem Fall kommt die Bezeichnung des neuen Datentyps nach den `{}`. Diese Methode funktioniert auch mit Strukturen, siehe nächstes Kapitel.

Anmerkung zu `typedef`

Synonyme mit `typedef` können auch bei »normalen« Variablen hilfreich sein. Zum Beispiel verwendet man beim Programmieren von Microcontrollern gerne Variablen mit genau definierten Breiten von 8, 16 und 32 Bit. Da die Definition von `int`, `long` usw. nicht standardisiert ist, ist auch die Breite unklar. Bei Microcontrollern verwendet man daher gerne folgende typedefs:

```
typedef unsigned char  uint8_t;
typedef unsigned short uint16_t;
typedef unsigned long  uint32_t;

void main(void)
{
    printf("Länge von uint8_t: %d\n", sizeof(uint8_t)); // 1 Byte
    printf("Länge von uint16_t: %d\n", sizeof(uint16_t)); // 2 Bytes
    printf("Länge von uint32_t: %d\n", sizeof(uint32_t)); // 4 Bytes
}
```

Dasselbe gibt es auch noch mit *signed*, also ohne `u`:

```
typedef char  int8_t;
typedef short int16_t;
typedef long  int32_t;
```

17 Strukturen

Mit Strukturen (engl. *structure*) können Variablen von verschiedenen Datentypen zusammengefasst werden. Strukturen können dann als neuer Datentyp verwendet werden.

17.1 Deklaration von Strukturen

Beispiel: Die Autos einer Firma sollen verwaltet werden; von den Autos sind folgende Daten festzuhalten:

- Marke
- Farbe
- Kennzeichen
- Anschaffungspreis

Marke, Farbe und Kennzeichen werden sinnvollerweise als Zeichenkette, der Anschaffungspreis als float abgespeichert.

Definition der Struktur:

```
struct fahrzeug
{
    char marke[21];
    char farbe[11];
    char kennzeichen[12];
    float preis;
};
```

Damit ist der neue Datentyp `struct fahrzeug` definiert. Man kann nun Variablen dieses Typs deklarieren:

```
struct fahrzeug wagen1, wagen2, wagen3; // 3 Variable des Typs
                                         // struct fahrzeug
```

Damit der Datentyp in jedem Unterprogramm verfügbar ist, wird er außerhalb der Unterprogramme am Anfang der Datei deklariert (nach `#define` ..., aber vor den Funktionen).

Wie bei `enum` ist es auch mit `struct` möglich, Synonyme für den neuen Datentyp anzugeben; dazu dient das Schlüsselwort `typedef` vor `struct`:

```
typedef struct fahrzeug
{
    char marke[21];
    char farbe[11];
    char kennzeichen[12];
    float preis;
} fahrzeug_t;
```

Damit entfällt bei der Variablen-Deklaration das `struct`:

```
fahrzeug_t wagen1, wagen2; // kein typedef davor
```

Möchte man nur mehr mit dem `typedef` arbeiten, so kann man den Namen `fahrzeug` oben auch weglassen (darum in grau). Für die Bezeichnung der `typedef`-Variable verwendet man gerne ein `_t`. `typedef` funktioniert auch bei `enum` und `union`, siehe weiter unten.

Initialisierung bei der Deklaration

Es ist auch bei `struct`-Variablen möglich, diese gleich bei der Deklaration zu initialisieren. Die Syntax ist ähnlich wie beim Array, nämlich mit `{}`:

```
fahrzeug_t wagen1 = { "Mercedes", "blau", "L-1234", 55500 };
```

17.2 Zugriff auf Strukturen

Um auf einzelne Variablen einer Struktur zuzugreifen, verwendet man den Punkt-Operator:

```
wagen1.preis = 100000;
```

Achtung: `wagen1.marke = "Audi"`; funktioniert nicht, weil ja strings in C nicht direkt zugewiesen werden können.

Beispiel:

```
void main(void)
{
    fahrzeug_t wagen1, wagen2;
    char marke[21];

    printf("Bitte Marke eingeben:\n");
    scanf_s("%20s", &wagen1.marke, 21);
    printf("Bitte Preis eingeben:\n");
    scanf_s("%f", &wagen1.preis);

    printf("Marke: %s\n", wagen1.marke);
    printf("Preis: %f\n", wagen1.preis);

    wagen2 = wagen1; /* Es können auch ganze structs kopiert werden! */

    printf("Marke: %s\n", wagen2.marke);
    printf("Preis: %f\n", wagen2.preis);
}
```

Es ist erstaunlich, dass in C ganze Strukturen, auch mit strings, durch einfache Zuweisung kopiert werden können! strings alleine können ja nicht kopiert werden.

Verschachtelung von Strukturen

Strukturen können Elemente von Strukturen sein.

```
typedef struct
{
    int tag;
    int monat;
    int jahr;
} datum_t;

typedef struct
{
    char marke[21];
    char farbe[11];
    char kennzeichen[12];
    float preis;
    datum_t zulassung;
} fahrzeug_t;
```

Zugriff auf die Struktur zulassung:

```
fahrzeug_t wagen1;
wagen1.zulassung.tag = 10;
wagen1.zulassung.monat = 4;
wagen1.zulassung.jahr = 2022;
```

Auch bei verschachtelten Strukturen ist eine Initialisierung bei der Deklaration möglich:

```
fahrzeug_t wagen1 = { "Mercedes", "blau", "L-1234", 55500,
    { 10, 4, 2022 } };
```

Zeiger auf Strukturen

Auch auf Elemente von Strukturen kann über Zeiger zugegriffen werden.

Beispiel:

```
fahrzeug_t wagen1 = { "Mercedes", "blau", "L-1234", 55500,
    { 10, 4, 2022 } };
fahrzeug_t* wagenZeiger; // Struktur-Zeiger deklarieren
```

```
wagenZeiger = &wagen1;           // Adresse zuweisen
(*wagenZeiger).preis = 54000.0;    // Zugriff auf preis
wagenZeiger->preis = 52000.0;      // Auch Zugriff auf preis

printf("Preis: %f\n", wagen1.preis); // Gibt 52000 aus
```

Für den Zugriff auf einen Struktur-Zeiger gibt es also einen eigenen Operator: `->`.

Feststellen der Größe einer Struktur

Manchmal benötigt man die Größe einer Struktur in Bytes. Man könnte dazu die Größen der einzelnen Elemente zusammenzählen. Das liefert jedoch nicht immer das richtige Ergebnis (der Compiler orientiert die einzelnen Komponenten unter Umständen in Gruppen von an 16/32/64 Bit)! Besser ist die Verwendung des `sizeof`-Operators:

```
fahrzeug_t wagen1;
printf("Gr\224\341e: %d Byte(s)\n", sizeof(wagen1));
printf("Gr\224\341e: %d Byte(s)\n", sizeof(fahrzeug_t));
```

`sizeof()` kann sowohl auf Variablen als auch auf Datentypen angewendet werden (siehe oben).

17.3 Übergabe von Strukturen an Unterprogramme

Es gelten die üblichen Regeln für Über- und Rückgabeparameter.

Beispiel:

```
void readWagen (fahrzeug_t* wagen)
{
    printf("Marke eingeben:\n");
    scanf_s("%20s", wagen->marke, 21);
    printf("Preis eingeben:\n");
    scanf_s("%f", &wagen->preis);
}

void main(void)
{
    fahrzeug_t wagen1;

    readWagen(&wagen1);

    printf("Marke: %s\n", wagen1.marke);
    printf("Preis: %f\n", wagen1.preis);
}
```

17.4 Aufgaben zu Strukturen

Aufgabe 1: Wagen

1. Erweitere das letzte Beispiel mit `readWagen()`, indem alle Parameter der Struktur `fahrzeug_t` eingelesen werden.
2. Schreib ein Unterprogramm `printfWagen()`, dass alle Parameter der Struktur `fahrzeug_t` ausgibt.

Aufgabe 2: Zeitunterschied

- Erstelle eine Struktur für die Uhrzeit mit Stunden, Minuten und Sekunden.
- Schreib ein Unterprogramm `readTime()`, welches eine Uhrzeit via Zeiger einliest.
- Schreib ein Unterprogramm `printTime()`, welches eine Uhrzeit im Format `hh:mm:ss` ausgibt.
- Lese zwei Uhrzeiten ein, bestimme mittels Unterprogramm `getDeltaTime()` die Differenz und gib die Differenz wieder im Format `hh:mm:ss` aus.

Aufgabe 3: Bruchrechnung

Es soll ein Programm zum einfachen Bruchrechnen erstellt werden. Verwende folgende Struktur für den Bruch:

```
typedef struct
{
    int numerator; // Zähler
    int denominator; // Nenner
}
```

```
} fraction_t;      // Bruch
```

Folgende Funktionen sollen implementiert werden:

```
/* Bruch einlesen */
fraction_t readFraction(void) { /* ... */ }

/* Bruch ausgeben */
void printFraction(fraction_t frac) { /* ... */ }

/* 2 Brüche addieren */
fraction_t addFraction(fraction_t summand1, fraction_t summand2) { /* ...
*/ }

/* 2 Brüche subtrahieren */
fraction_t subFraction(fraction_t subtrahend1, fraction_t subtrahend2) {
/* ... */ }

/* Multiplizieren von Brüchen */
fraction_t multFraction(fraction_t multiplicand, fraction_t multiplier) {
/* ... */ }

/* Dividieren von Brüchen */
fraction_t divFraction(fraction_t dividend, fraction_t divisor) { /* ...
*/ }

/* Kürzen */
fraction_t cancelFraction(fraction_t frac) { /* ... */ }
```

Überlege dir die jeweiligen Rechenschritte mit Hilfe der beiden Brüche $\frac{a}{b}$ und $\frac{c}{d}$. Für das Kürzen der Brüche bietet sich eine der ggT-Funktionen an. Beachte hier auch mögliche negative Zahlen in den Brüchen!

Schreibe ein geeignetes Hauptprogramm zum Testen der Funktionen.

Erweiterung: verbessere die Ausgabe des Bruchs, indem ermittelt wird, wie viele Stellen Zähler und Nenner haben und dann ein geeigneter Bruchstrich gezeichnet wird. Zähler und Nenner sollen in etwa zentriert ausgegeben werden. Bei negativem Vorzeichen soll das Minus vor dem Bruchstrich ausgegeben werden.

18 Dateizugriffe

Es gibt verschiedene Arten von Dateizugriffen. Wir wollen uns auf den Zugriff von Textdateien beschränken.

Beispiel zum Öffnen einer Datei zum Schreiben:

```
#include <stdio.h>

void main(void)
{
    FILE* file;    // FILE ist eine komplexe Struktur aus stdio.h
    int err;       // Variable für mögliche Fehler

    err = fopen_s(&file, "Textfile.txt", "w"); // Öffnen zum Schreiben

    if (err)       // Fehler, falls != 0
    {
        printf("Fehler beim Anlegen der Datei!\n");
    }
    else
    {
        /* Datei hier befüllen */
        fclose(file);
    }
}
```

Die Datei `Testfile.txt` wird im Projektverzeichnis erzeugt. Falls sie bereits vorhanden ist, wird die vorhandene Datei überschrieben. Die Parameter von `fopen_s()` sind:

1. Ein Pointer auf einen FILE-Pointer. Darum noch einmal ein `&`, obwohl `file` ja schon ein Pointer ist.
2. Der Dateiname. Hier können auch Laufwerke und Verzeichnisse eingegeben werden, z. B. `"C:\\1\\Textfile.txt"` bzw. `"C:/1/Textfile.txt"`. Achtung: das Verzeichnis muss dazu schon existieren.
3. Die Art des Dateizugriffs. Details siehe weiter unten.

`fclose()` ist sehr wichtig. Damit werden die Daten tatsächlich in die Datei geschrieben und die Datei wird für das Betriebssystem freigegeben.

Beispiel zum Öffnen einer Datei zum Lesen:

```
#include <stdio.h>

void main(void)
{
    FILE* file;    // FILE ist eine komplexe Struktur aus stdio.h
    int err;       // Variable für mögliche Fehler

    err = fopen_s(&file, "Textfile.txt", "r"); // Öffnen zum Lesen

    if (err)       // Fehler, falls != 0
    {
        printf("Fehler beim Öffnen der Datei!\n");
    }
    else
    {
        /* Datei hier einlesen */
        fclose(file);
    }
}
```

Der Unterschied liegt im Grunde nur in der Art des Dateizugriffs. Falls die Datei nicht existiert, erfolgt eine Fehlermeldung.

Viele Funktionen, die bereits von der Bildschirmausgabe bzw. der Manipulation von strings bekannt sind, gibt es in ähnlicher Weise auch für den Dateizugriff:

Funktion	Was macht die Funktion?
ch = fgetc(file)	Liest ein Zeichen aus der Datei ein
fgets(str, len, file);	Liest eine Zeile bzw. max. len Zeichen aus der Datei ein
fscanf_s(file, "%d", &var);	Liest Zahlen etc. gemäß Format aus der Datei ein
fputc(ch, file);	Schreibt ein Zeichen in die Datei
fputs(str, file);	Schreibt einen string in die Datei
fprintf(file, "%d", var);	Schreibt Zahlen etc. gemäß Format in die Datei

Öffnungsmodi bei Dateizugriffen

Der Öffnungsmodus ist der 3. Parameter von `fopen_s()` (Achtung: Parameter ist Zeichenkette angeben).

Mögliche Modi:

Parameter	Bedeutung
r	Nur lesen, die Datei muss existieren
w	Nur schreiben; eine eventuell bestehende Datei wird überschrieben
a	<i>append</i> ; Schreiben am Ende einer bestehenden Datei, bzw. Erzeugen einer neuen Datei.
r+	Lesen und schreiben, die Datei muss existieren
w+	Lesen und schreiben, die Datei wird neu erstellt
a+	Lesen und schreiben (schreiben am Dateiende)

Erfolgt nach dem Modus ein b, so wird die Datei im Binär-Format behandelt, z. B. "rb".

Weitere Funktionen für Dateizugriffe

Funktion	Was macht die Funktion?
feof(file);	Liefert true am Ende der Datei Beispiel: <pre>while (!feof(file)) { fgets(str, MAX_STR_LEN, file); printf("%s", str); }</pre>
ferror(file);	Liefert -1, wenn bei einer Operation mit file ein Fehler aufgetreten ist
fflush(file);	Schreibt den Puffer in die Datei (ohne sie zu schließen)
rewind(file);	Setzt die Position, an der geschrieben wird, auf den Datei-Anfang zurück

Beispiel: Schreiben und Lesen einer Text-Datei

```
#include <stdio.h>
#define MAX_STR_LEN    50

void main(void)
{
    FILE* file;
    int err;
    char str[MAX_STR_LEN];
    int intFromFile;
    int intToFile = 222;

    err = fopen_s(&file, "higscore.txt", "w"); // Öffnen zum Schreiben

    if (err)
    {
        printf("Fehler beim Anlegen der Datei!\n");
    }
    else
    {
        fprintf(file, "Player 1\n");
        fprintf(file, "%d", intToFile);
        fclose(file);
    }

    /* ... */
}
```

```

err = fopen_s(&file, "higscore.txt", "r"); // Öffnen zum Lesen

if (err)
{
    printf("Fehler beim Öffnen der Datei!\n");
}
else
{
    fgets(str, MAX_STR_LEN, file);
    fscanf_s(file, "%d", &intFromFile);
    printf("Name: %s", str);
    printf("High Score: %d\n", intFromFile);
    fclose(file);
}
}

```

Aufgabe 1: Erweiterung High Score

Erweitere dieses Beispiel um die Möglichkeit, 4 weitere Spieler mit Punktzahl durch Tastatureingabe zu ergänzen und anschließend am Bildschirm auszugeben. Erweitere das Schreiben mit einer `for`-Schleife und das Lesen mit einer `while(!feof(fp))`-Schleife.

Aufgabe 2: Winkelfunktionen

Schreibe ein Programm, welches eine Tabelle der Winkelfunktionen $\sin(x)$, $\cos(x)$, $\tan(x)$ in eine Datei ausgibt. Gib in der 1. Spalte x selbst aus. x soll von 0 bis 1 in Schritten von 0,01 gehen und dabei eine vollständige Periode durchlaufen (2π). Gib zu Beginn auch eine Überschrift-Zeile aus. Importiere die Daten in Excel, Calc, o. ä. und prüfe, ob die Berechnung richtig ist.

Aufgabe 3: Textanalysator

Erstelle ein Programm, das eine Datei `text.txt` analysiert. Gib dazu eine Liste aller Buchstaben A–Z und der Anzahl der Vorkommnisse in der Datei aus. Unterscheide nicht zwischen Groß- und Kleinbuchstaben. Sonstige Zeichen sollen ignoriert werden. Gib die Liste am Bildschirm und in einer eigenen Datei `analyse.txt` aus.

Aufgabe 4: b-Sprache revisited

Benutze das Programm mit der b-Sprache, das wir im string-Kapitel geschrieben haben und wende es auf eine Datei `text.txt` an. Gib den b-Sprachentext in einer neuen Datei `bsprachetext.txt` aus.

Aufgabe 5: Geheimsprache

Erstelle ein Programm, dass bei einem gegebenen Textfile `top_secret.txt` jede Zeile invertiert, sowie den Buchstaben `e` durch `x` ersetzt. Schreibe das Ergebnis in die neue Datei `code.txt`.

Beispiel:

```
Hallo Welt!
Wie geht es?
```

Wird zu:

```
!tlxW ollaH
?sx thxg xiW
```

Gib das Ergebnis in der Datei `code.txt` auch am Bildschirm aus.

19 Bitverarbeitung

Bisher haben wir immer mit Variablen gearbeitet, die sich aus vielen Bits zusammen setzen. Oft ist es erforderlich, dass nur bestimmte Bits geprüft, verändert, ... werden. In diesem Kapitel verwenden wir Variablen vom Typ `unsigned char`.

Beispiel: die Dezimalzahl 91 entspricht der Binärzahl 01011011. Die Nummerierung der Bits beginnt rechts mit 0 (1. Bit, LSB = *least significant bit*) und geht bis 7 (8. Bit, MSB = *most significant bit*):

Bit	7	6	5	4	3	2	1	0
Ziffer	0	1	0	1	1	0	1	1

19.1 Verknüpfungen von Bits

Einzelne Bits können mit verschiedenen binären Operationen verknüpft werden:

A	B	A und B	A	B	A oder B	A	B	A xor B	A	nicht A
0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	1	1	0	1	1	1	0
1	0	0	1	0	1	1	0	1		
1	1	1	1	1	1	1	1	0		

Aufgabe 1

Angenommen, eine Variable *a* habe den Binärwert 00001010 und eine Variable *b* den Binärwert 00000011: berechne das Ergebnis *c* der jeweiligen Verknüpfung.

- UND-Verknüpfung: $c = a \& b;$ $c =$ _____
- ODER-Verknüpfung: $c = a \mid b;$ $c =$ _____
- XOR-Verknüpfung: $c = a \wedge b;$ $c =$ _____
- Nicht (1er Komplement): $c = \sim a;$ $c =$ _____

Hinweis: achte darauf, dass die binären Operatoren (&, |) nicht mit den logischen Operatoren (&&, ||) verwechselt werden!

Aufgabe 2

Was ist das Ergebnis von $c = a \&\& b;$?

19.2 Schiebeoperationen

Mit den Schiebeoperatoren (engl. *shift operator*) können alle Bits einer Variable in der Position verschoben werden.

Left shift

Mit dem Operator << können alle Bits nach links verschoben werden.

Beispiel:

```
unsigned char a = 0x35; // = 00110101b = 53d
unsigned char b = a << 1; // = 01101010b = 106d
unsigned char c = a << 3; // = 10101000b = 168d
unsigned char d = 1 << 3; // = 00001000b = 8d
```

Solange es keine Überlauf gibt, entspricht 1 shift nach links einer (Ganzzahl-)Multiplikation von 2. Die neuen Stellen rechts werden mit 0 befüllt.

Right shift

Mit dem Operator >> können alle Bits nach rechts verschoben werden.

Beispiel:

```
unsigned char a = 0x35; // = 00110101b = 53d
unsigned char b = a >> 1; // = 00011010b = 26d
```

Ein shift nach rechts entspricht einer (Ganzzahl-)Division von 2. Die neuen Stellen links werden mit 0 befüllt.

Achtung: das Shiften von unsigned-Variablen ist unproblematisch. Schwierig wird das Verhalten bei signed-Variablen; es hängt vom Compiler ab, wie mit dem Vorzeichen-Bit ganz links umgegangen wird. Es ist besser, keine shift-Operationen mit signed-Variablen durchzuführen.

19.3 Beispiele mit Bitoperationen

- Ein Byte auf 0 setzen: $b = 0$; oder $b = b \wedge b$;
- Das k . Bit setzen, Rest 0: $b = 1 \ll k$;
- Das k . Bit setzen, Rest unverändert: $b |= (1 \ll k)$;
- Das k . Bit löschen, Rest unverändert: $b \&= \sim(1 \ll k)$;
- Das k . Bit toggeln (invertieren): $b \wedge= (1 \ll k)$;

19.4 Ein Bit abfragen

Das wird vor allem bei der Programmierung von Microcontrollern benötigt, wenn z. B. einzelne Port-Pins ausgelesen werden sollen.

Beispiel: prüfen, ob ein bestimmtes Bit gesetzt wurde bzw. gewünschtes Bit erhalten

```
#define DESIRED_BIT 2

void main(void)
{
    unsigned char a = 0x0e;      // = 00001110b = 14d
    unsigned char bitOnly;

    if (a & (1 << DESIRED_BIT)) // != 0, wenn gesetzt
    {
        /* Bit gesetzt */
    }

    bitOnly = (a & (1 << DESIRED_BIT)) >> DESIRED_BIT;
    /* 1, wenn gesetzt, 0 wenn nicht gesetzt */
}
```

Aufgabe 1

Schreibe ein Unterprogramm `printBin()`, welches eine gegebene Zahl (`unsigned char`) als Binärzahl ausgibt.

Aufgabe 2

Setze beim Beispiel oben das 3. Bit von `a` auf 1, ohne die übrigen Bits zu verändern. Prüfe das Ergebnis.

Aufgabe 3

Setze das 3. Bit von `a` wieder auf 0, ohne die übrigen Bits zu verändern. Prüfe das Ergebnis.

Aufgabe 4

Schreibe ein Programm, dass einen Türkontakt simuliert. Falls die Tür offen ist, soll das 5. Bit von `a` 0 sein und sonst 1. Gib die Meldung »Die Tür ist offen« aus, wenn das 5. Bit 0 ist. Simuliere beide Fälle, indem du das 5. Bit setzt/löscht.

20 Dynamische Speicherverwaltung

20.1 Speicher anlegen und freigeben

Wir haben in Kapitel 14 Felder und in Kapitel 15 strings kennen gelernt. In beiden Fällen wurden dazu Felder mit einer konstanten Größe angelegt. Benötigt man beispielsweise einen string für einen Namen, so muss sicherheitshalber ein relativ großes Feld initialisiert werden, um auch lange Namen speichern zu können.

Mit der Bibliothek `stdlib.h` stehen Funktionen zur Verfügung, um dynamisch Speicher zu reservieren. Man spricht auch davon, Speicher zu *allozieren* (engl. *memory allocation*). Der Vorteil dynamischer Speicherverwaltung ist, dass wirklich nur so viel Speicher verwendet werden muss, wie auch tatsächlich benötigt wird.

Eine wichtige Funktion ist `malloc()` (eben *memory allocation*). Hier ein Beispiel:

```
#include <stdlib.h>

void main (void)
{
    int *data;
    int len = 100;

    // Speicher allozieren
    data = (int *) malloc(len * sizeof(int));
}
```

Damit wird dynamisch eine Array mit einer Größe von 100 ints angelegt⁷. Beachte, dass `malloc()` die Größe des Arrays in Byte benötigt, darum der `sizeof`-Operator. Da mit `malloc()` unterschiedliche Datentypen alloziert werden können, ist der Rückgabewert vom Typ `void *`. Das ist kein Zeiger auf das »Nichts«, sondern man sagt besser *Zeiger auf einen unbekannten Datentyp* dazu. Um daraus z. B. einen Integer zu machen wird mittels `(int *)` gecastet.

Nach dem Allozieren kann die Variable `*data` wie ein normales Feld behandelt werden, z. B. `data[0] = 123`;. Wichtig bei der dynamischen Speicherverwaltung ist, den Speicher wieder freizugeben, sobald er nicht benötigt wird. Dazu verwendet man die Funktion `free()`. Ein etwas vollständigeres Beispiel:

```
#include <stdlib.h>
#include <stdio.h>

void main(void)
{
    int* data;
    int len;
    int i;

    printf("Wie viele Variablen werden ben\224tigt: ");
    scanf_s("%d", &len);

    // Speicher allozieren
    data = (int*) calloc(len, sizeof(int));

    if (data == NULL)
    {
        printf("Fehler beim Allozieren des Speichers!\n");
    }
    else
    {
        for (i = 0; i < len; i++)
        {
            // Daten verwenden ...
            data[i] = 0;
        }

        // Speicher wieder freigeben
        free(data);
    }
}
```

⁷ Man könnte jetzt sagen, `len` ist ja eine Konstante. Es funktioniert aber auch, indem man `len` z. B. vorher per `scanf_s()` einliest.

Beachte, dass mittels Vergleich mit NULL (oder 0) geprüft werden kann, ob der Speicher auch tatsächlich alloziert wurde. Das wird empfohlen, da es auch passieren kann, dass kein Speicher mehr frei ist.

Die Funktion `malloc()` reserviert lediglich Speicher, initialisiert ihn jedoch nicht. Um Speicher mit dem Wert 0 zu initialisieren gibt es die Funktion `calloc()`. Diese Funktion benötigt zwei Übergabeparameter: 1. die Feld-Länge und 2. die Variablen-Größe in Byte:

```
data = (int*) calloc(len, sizeof(int));
```

Das dynamische Allokieren von Speicher funktioniert natürlich auch mit Strukturen. Dazu kann man die Größe der Struktur mittels `sizeof(mystruct_t)` bestimmen.

Aufgabe 1: Notenbestimmung revisited

Schreibe ein Programm, welches die Anzahl der Schüler einer Klasse einliest. Lese dann einzelne Noten ein und speichere sie dynamisch. Gib den Notenspiegel und den Notendurchschnitt aus.

Aufgabe 2: Wagen revisited

Implementiere noch einmal die Aufgabe 1 aus Abschnitt 17.4. Frage zunächst, wie viele Wagen angelegt werden sollen. Befülle dann die gegebene Anzahl mit Daten. Gib anschließend alle eingegebenen Daten aus.

20.2 Größe des Speichers verändern

Man kann die Speichergröße auch zur Laufzeit verändern. Dazu wird die Funktion `realloc()` (re-allocate) verwendet. Die Verwendung sieht so aus:

```
data = (int*) realloc(data, newlen * sizeof(int));
```

Es wird der ursprüngliche Zeiger und die neue Länge des Felds benötigt. Der Rückgabeparameter ist wieder ein Zeiger auf den veränderten Speicherbereich.

Vorsichtig muss man sein, wenn man die Speichergröße in einem Unterprogramm verändern möchte und `data` dabei an das Unterprogramm übergeben wird. Beispiel, wie es nicht funktioniert:

```
void addNumber(int* mydata, int* len)
{
    *len = *len + 1;
    mydata = (int*) realloc(mydata, *len * sizeof(int));
}
```

Hier wird versucht, die übergebene Länge um 1 zu erhöhen und einen Integer an das bestehende Feld anzuhängen. Es kann jedoch vorkommen, dass `realloc()` keinen freien Speicher mehr am Ende des bestehenden Felds findet. Dann muss das gesamte Feld in einen Speicherbereich verlegt werden, der genug Platz hat. Damit das funktioniert, ist es besser, das re-allozierte Feld als neuen Pointer zurückzugeben:

```
int* addNumber(int* mydata, int* len)
{
    int* newData;
    *len = *len + 1;
    newData = (int*) realloc(mydata, *len * sizeof(int));
    return newData;
}
```

Im Hauptprogramm weist man dann den Rückgabeparameter dem ursprünglichen Feld zu, also

```
data = addNumber(data, &len);
```

Warum ist das so? Weil durch die Übergabe eines Pointers an das Unterprogramm zwar die Daten des Pointers bzw. die Daten des Arrays verändert werden können, aber nicht der Pointer selbst, also die Adresse, auf die gezeigt wird. Um dieselbe Funktionalität ohne Rückgabe als neuen Pointer zu realisieren, müsste deshalb ein *Pointer auf einen Pointer* übergeben werden (`int**`). Das ist möglich und wird z. B. auch beim Dateizugriff (Kapitel 18, `fopen_s`) insgeheim verwendet. Hier wird es aber relativ rasch relativ kompliziert!

`realloc()` lässt die Daten des ursprünglichen Arrays unverändert. Muss der Speicher tatsächlich verlegt werden, so kopiert die Funktion die alten Daten an die neue Stelle. Das ist gut so, aber benötigt mitunter etwas Zeit. Falls man sehr oft re-allozieren muss, dann bietet sich an, immer größere Pakete auf einmal anzulegen.

Aufgabe 3: Wagen noch einmal

Erweitere Aufgabe 2, indem du eine Funktion `fahrzeug_t* addWagen(fahrzeug_t* data, int* len)` hinzufügst. In dieser Funktion soll ein neues Fahrzeug eingelesen werden (`readWagen()`) und an die bestehenden Fahrzeuge angehängt werden. Beachte das Kopieren von strings!